



PUC Minas

ÁRVORE BINÁRIA

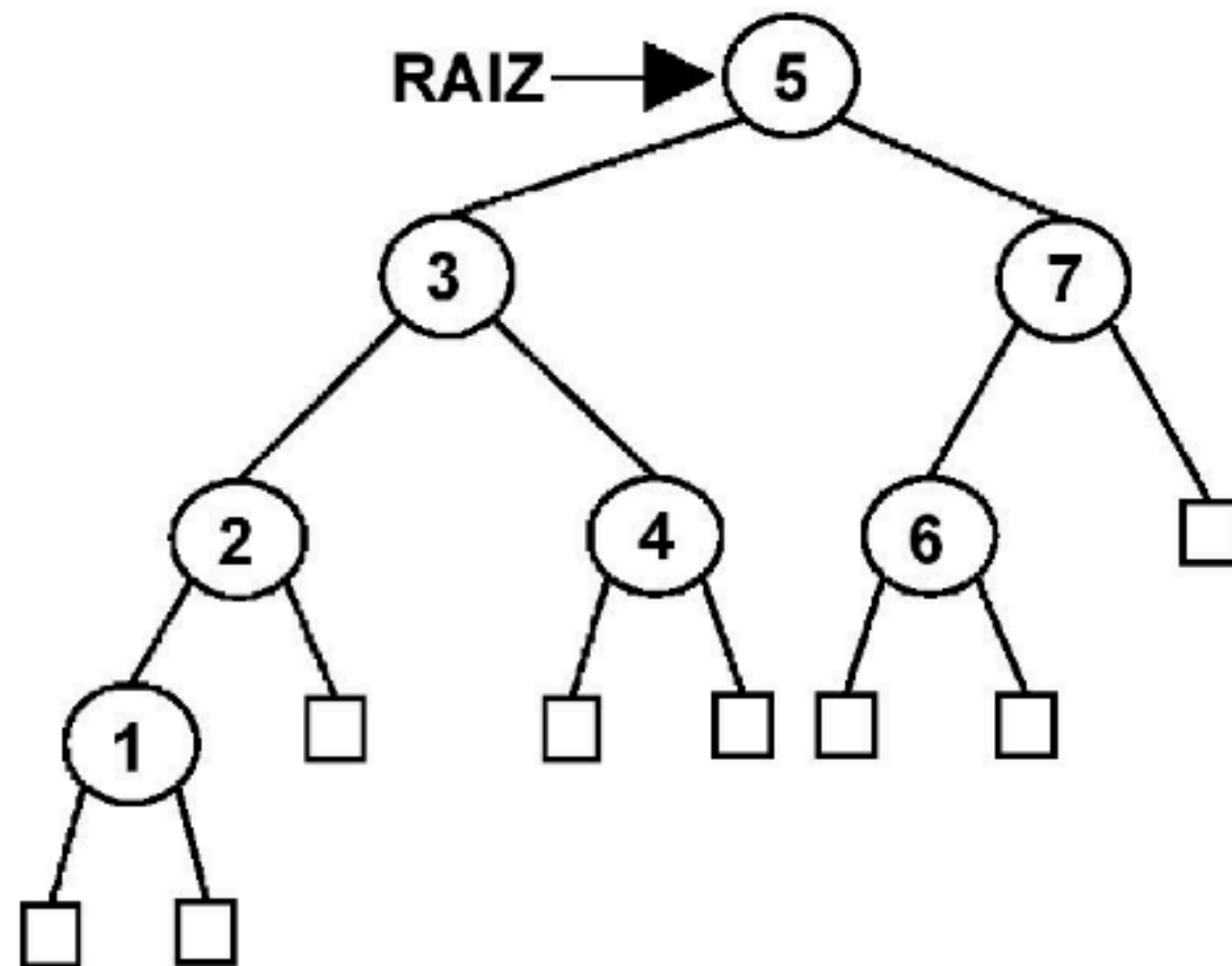
AULÃO DE AEDS 2



Tópicos

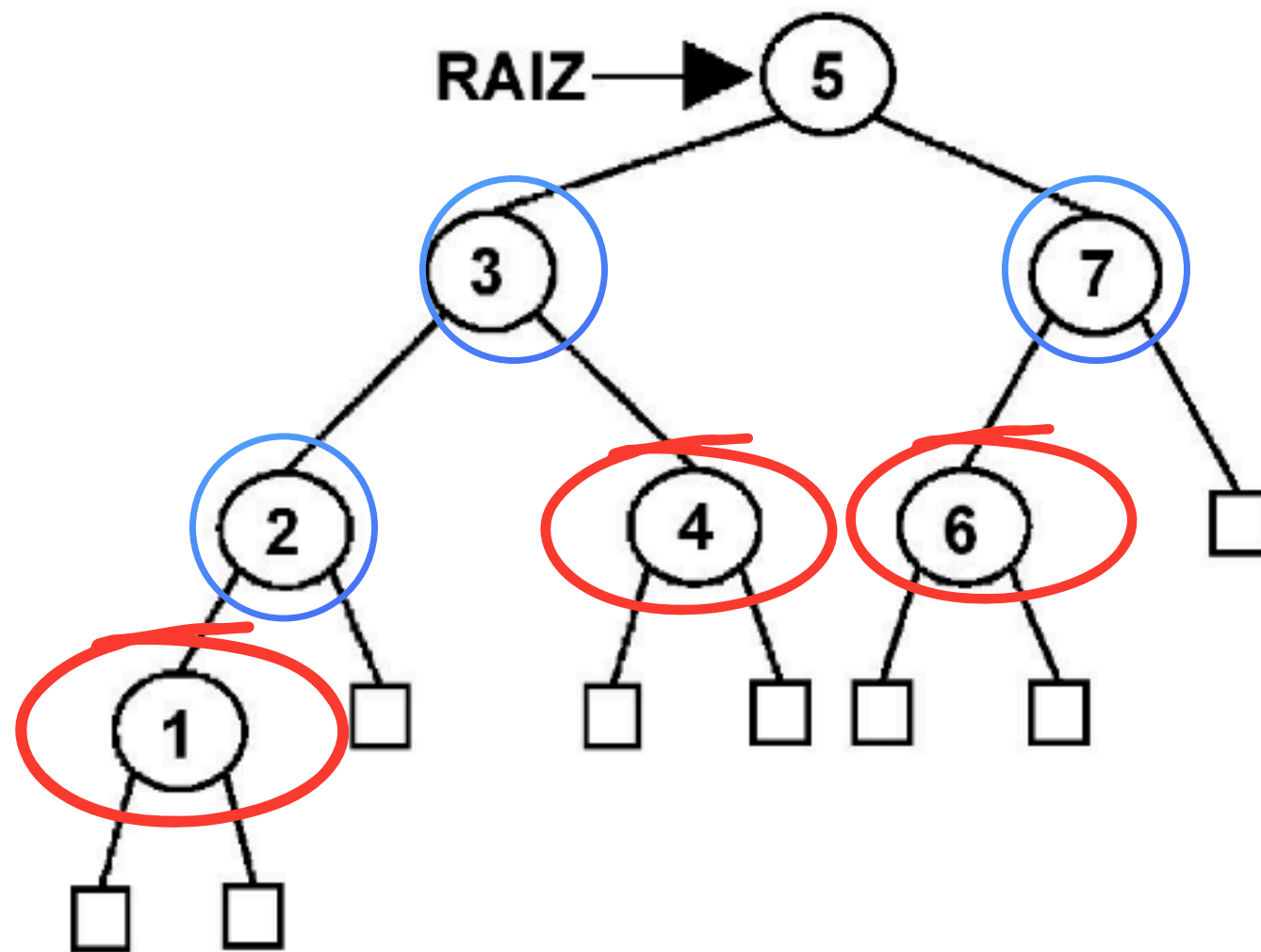
- Árvore Binária
- Árvore Binária Completa
- Árvore Binária de Busca
- Custo
- Inserir Java
- Inserir C
- Pesquisar
- Caminhar

Árvore Binária



- Em uma árvore binária todo nó possui **no máximo dois filhos**.
- Nós **sem filhos** são chamados de **folhas**.
- Nós **com filhos** são chamados de **nós internos**.
- Toda árvore tem somente **uma raiz**.

Árvore Binária



- Em vermelho estão os nós folha
- Em azul os nós internos

- Em uma árvore binária todo nó possui **no máximo dois filhos**.
- Nós **sem filhos** são chamados de **folhas**.
- Nós **com filhos** são chamados de **nós internos**.
- Toda árvore tem somente **uma raiz**.
- **Altura**: maior distância entre um nó e a raiz.

Regras AB. COMPLETA

REGRA 1

Cada nó é uma folha ou possui exatamente dois filhos.

REGRA 2

Todos os nós possuem a mesma altura.



Regras AB. DE BUSCA

REGRA 1

O filho da direita deve ser sempre maior que o pai, e o filho da esquerda deve ser sempre menor que o pai.

REGRA 2

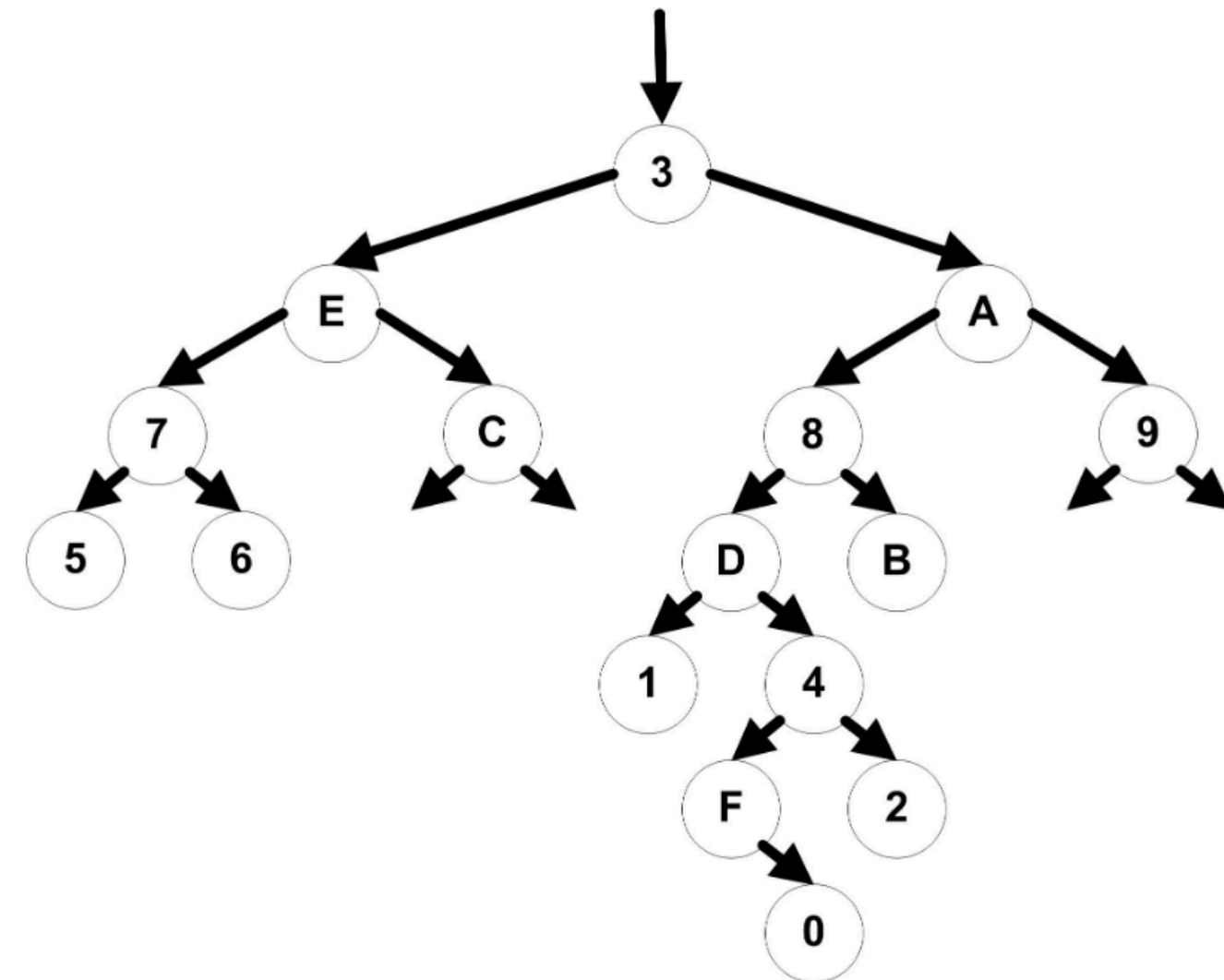
Todo nó tem no máximo dois filhos.



- Crie uma árvore com os dígitos hexadecimais (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E e F)



- Crie uma árvore com os dígitos hexadecimais (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E e F)



Benefícios

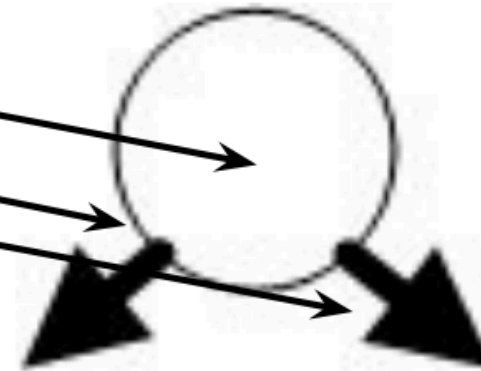
O custo **pode** ser (se a árvore estiver balanceada)

- Custo de Pesquisa: $O(\log n)$.
- Custo de Inserção: $O(\log n)$.
- Custo de Pesquisa: $O(\log n)$.



Classe Nó

```
class No {  
  int elemento;  
  No esq;  
  No dir;  
  No(int elemento) {  
    this(elemento, null, null);  
  }  
  No(int elemento, No esq, No dir) {  
    this.elemento = elemento;  
    this.esq = esq;  
    this.dir = dir;  
  }  
}
```



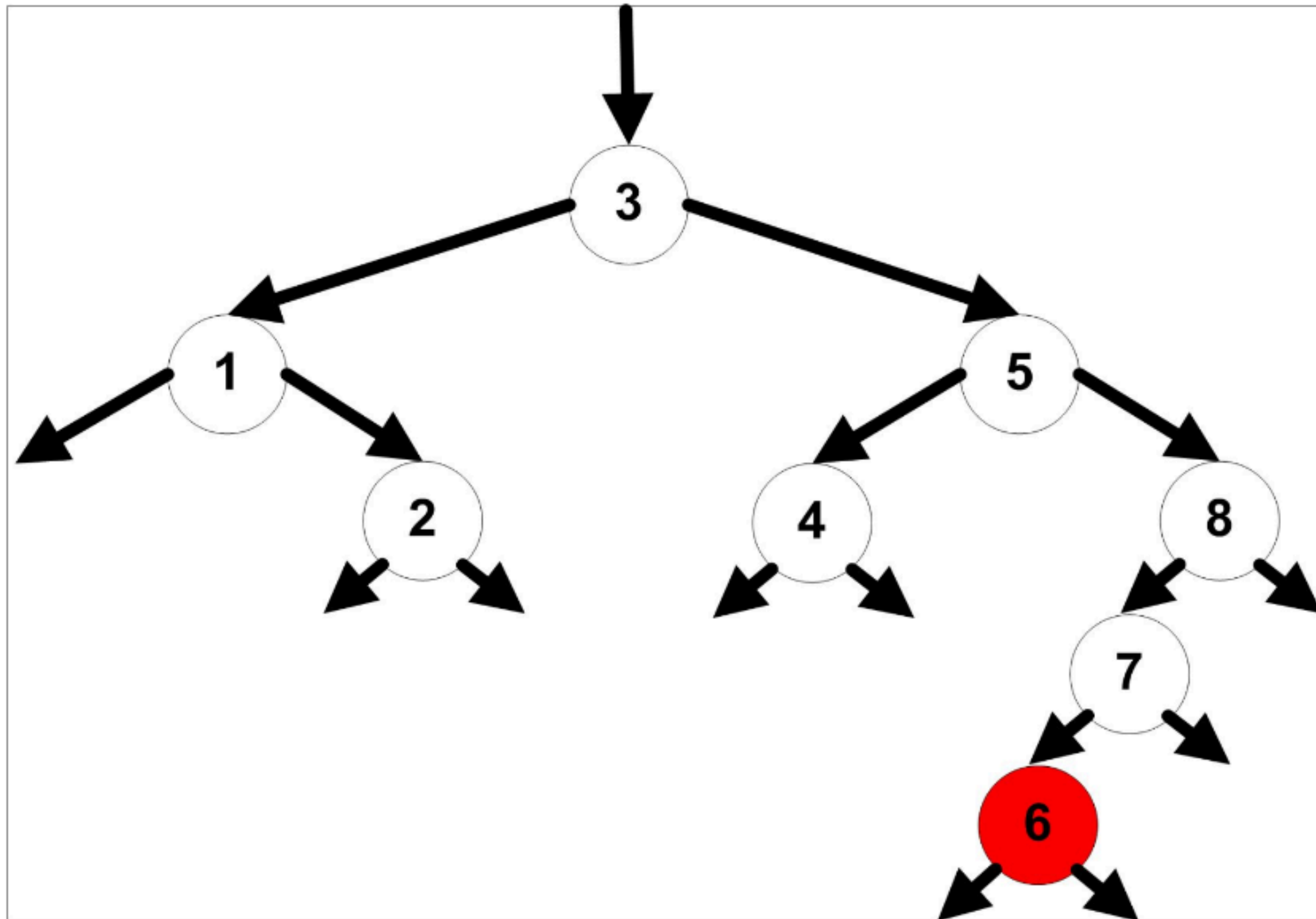
Classe Árvore

```
class ArvoreBinaria {  
    No raiz;  
    ArvoreBinaria() { raiz = null; }  
    void inserir(int x) { }  
    void inserirPai(int x) { }  
    boolean pesquisar(int x) { }  
    void caminharCentral() { }  
    void caminharPre() { }  
    void caminharPos() { }  
    void remover(int x) { }  
}
```

- Inserir, na ordem, os elementos 3, 5, 1, 8, 2, 4, 7 e 6



- Inserir, na ordem, os elementos 3, 5, 1, 8, 2, 4, 7 e 6



Inserir

Podemos pensar no inserir como uma **série de if's**

- 1) Se a raiz estiver **vazia** inserimos o elemento nela.
- 2) **Senão**, se o elemento for menos do que a raiz nos movemos para a subárvore da esquerda.
- 3) **Senão**, se o elemento for maior que o da raiz nos movemos para a subárvore da direita.
- 4) **Senão**, se o novo elemento for igual ao da raiz, **não inserimos o elemento repetido**.



Inserir

```
void inserir(int x) throws Exception {
```

```
    raiz = inserir(x, raiz);  
}
```

```
No inserir(int x, No i) throws Exception {
```

```
    if (i == null) {
```

```
        i = new No(x);
```

```
    } else if (x < i.elemento) {
```

```
        i.esq = inserir(x, i.esq);
```

```
    } else if (x > i.elemento) {
```

```
        i.dir = inserir(x, i.dir);
```

```
    } else {
```

```
        throw new Exception("Erro!");
```

```
    }
```

```
    return i;
```

```
}
```


Inserir em C

- Faça o quadro de memória e mostre a saída n

```
int x = 10;  
  
int * y = & x;  
  
printf("\n%i", x);  
  
printf("\n%p", &x);  
  
printf("\n%p", y);  
  
printf("\n%p", &y);  
  
printf("\n%i", *y);
```

Memória		
x	10	75h
		...
y	75h	CDh

Inserir em C

- Faça o quadro de memória e mostre a saída na tela:

```
int x = 10;
```

```
int * y = & x;
```

```
printf("\n%i", x);
```

```
printf("\n%p", &x);
```

```
printf("\n%p", y);
```

```
printf("\n%p", &y);
```

```
printf("\n%i", *y);
```

	Memória	
x	10	75h
		...
y	75h	CDh

Tela
10
75h
75h
CDh
10

Nó em C

```
//no.h
```

```
typedef struct No {  
    int elemento;  
    struct No *esq, *dir;  
} No;
```

```
No* novoNo(int);
```

```
//no.c
```

```
#include <stdlib.h>  
#include "no.h"
```

```
No* novoNo(int elemento) {  
    No* novo = (No*) malloc(sizeof(No));  
    novo->elemento = elemento;  
    novo->esq = NULL;  
    novo->dir = NULL;  
    return novo;  
}
```

Inserir

//código em Java

```
void inserir(int x) {  
    raiz = inserir(x, raiz);  
}  
  
No inserir(int x, No i) {  
    if (i == null) {  
        i = new No(x);  
    } else if (x < i.elemento) {  
        i.esq = inserir(x, i.esq);  
    } else if (x > i.elemento) {  
        i.dir = inserir(x, i.dir);  
    } else {  
        throw new ("Erro!");  
    }  
    return i;  
}
```

//código em C

```
void inserir(int x) {  
    raiz = inserirRec(x, raiz);  
}  
  
No* inserirRec(int x, No* i) {  
    if (i == NULL) {  
        i = novoNo(x);  
    } else if (x < i->elemento) {  
        i->esq = inserirRec(x, i->esq);  
    } else if (x > i->elemento) {  
        i->dir = inserirRec(x, i->dir);  
    } else {  
        errx(1, "Erro ao inserir!");  
    }  
    return i;  
}
```


PESQUISAR

Para a pesquisa na árvore binária de busca, **obrigatoriamente** os filhos da esquerda serão menores, e os filhos da direita serão maiores.

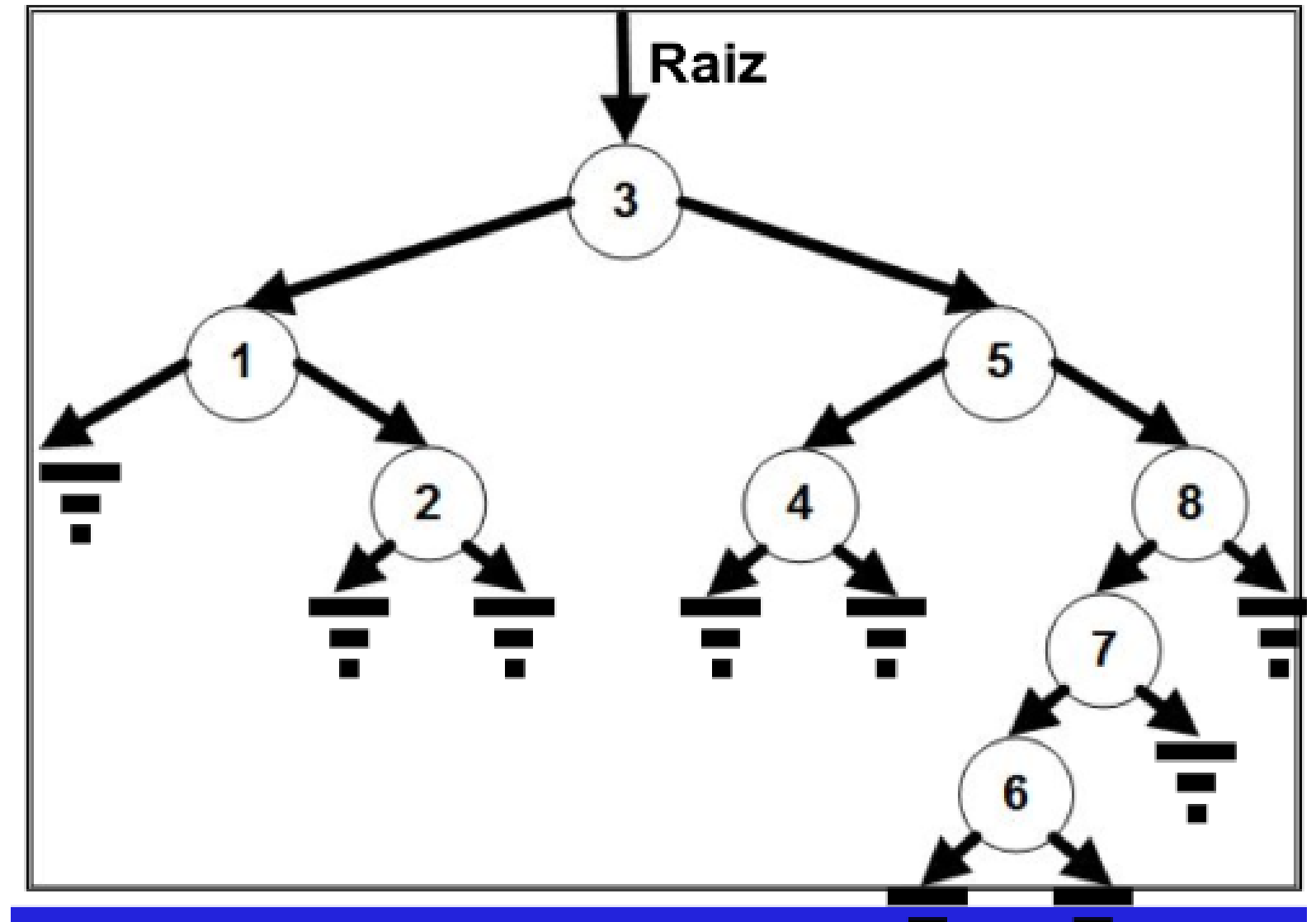


PESQUISAR

```
//Pesquisar(4)
```

```
boolean pesquisar(int x) {  
    return pesquisar(x, raiz);  
}
```

```
boolean pesquisar(int x, No i) {  
    boolean resp;  
    if (i == null) {  
        resp = false;  
    } else if (x == i.elemento) {  
        resp = true;  
    } else if (x < i.elemento) {  
        resp = pesquisar(x, i.esq);  
    } else {  
        resp = pesquisar(x, i.dir);  
    }  
    return resp;  
}
```

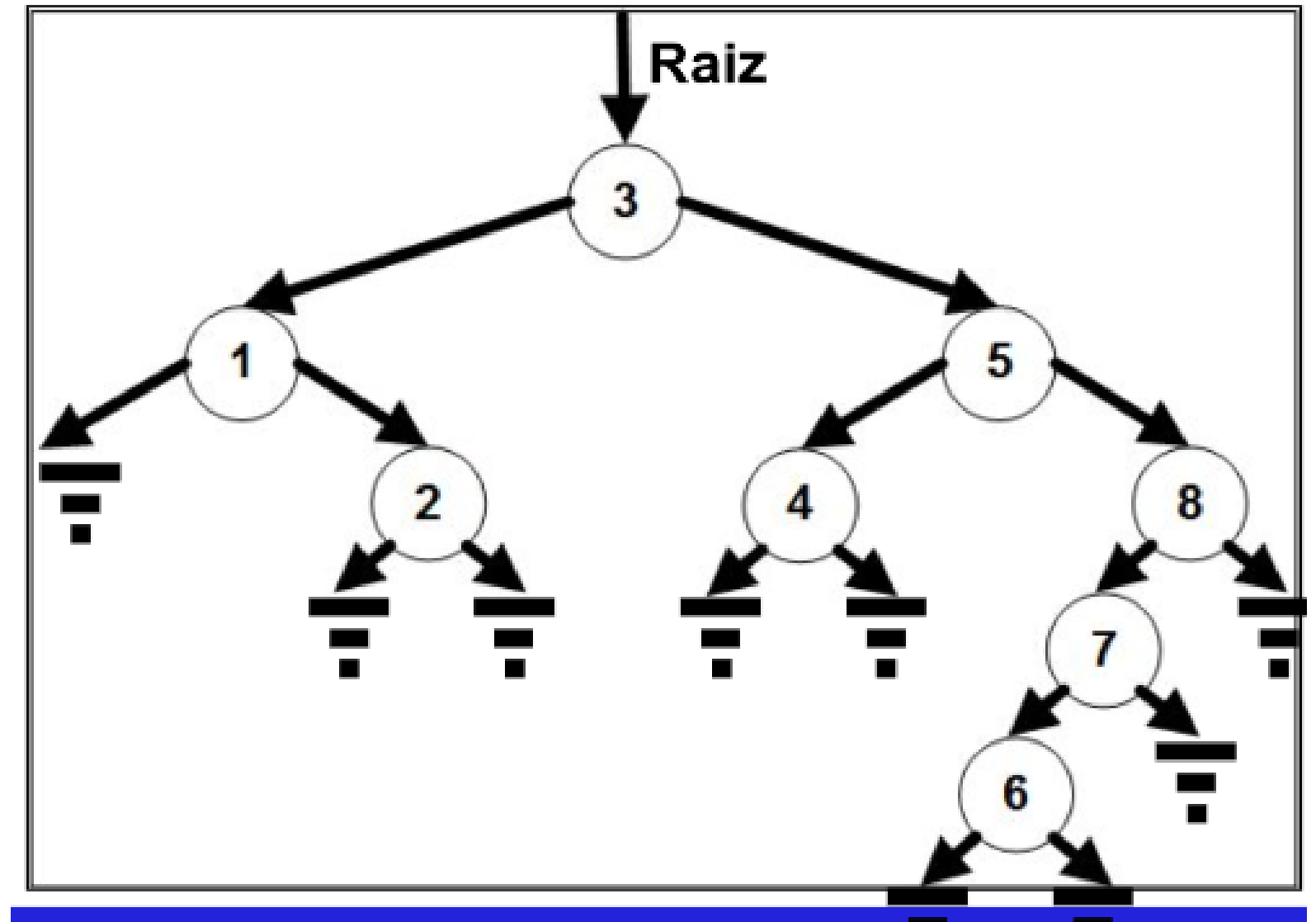


PESQUISAR

```
//Pesquisar(4)
```

```
boolean pesquisar(int x) {  
    return pesquisar(x, raiz);  
}
```

```
boolean pesquisar(int x, No i) {  
    boolean resp;  
    if (i == null) {  
        resp = false;  
    } else if (x == i.elemento) {  
        resp = true;  
    } else if (x < i.elemento) {  
        resp = pesquisar(x, i.esq);  
    } else {  
        resp = pesquisar(x, i.dir);  
    }  
    return resp;  
}
```



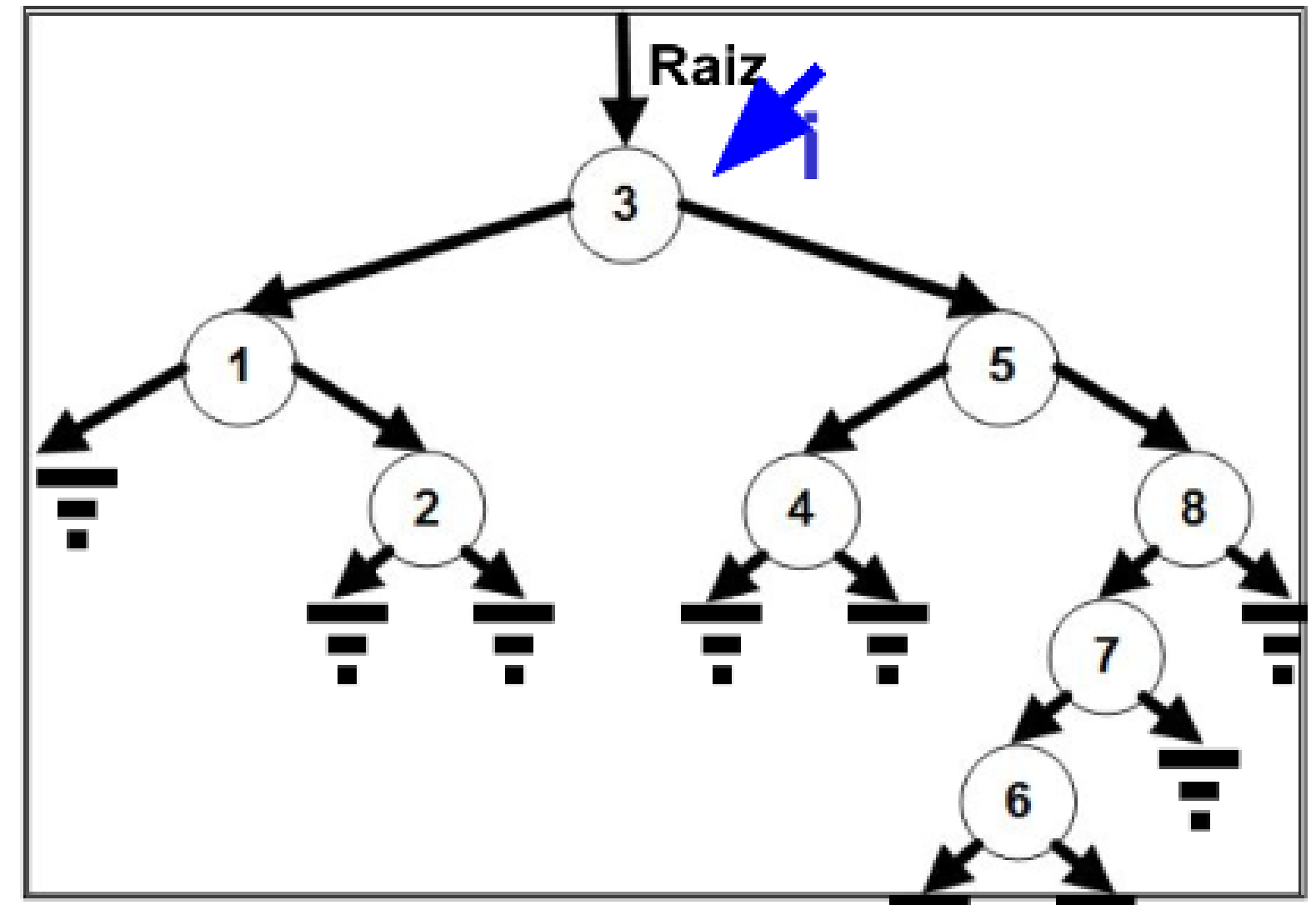
PESQUISAR

```
//Pesquisar(4)
```

```
boolean pesquisar(int x) {  
    return pesquisar(x, raiz);  
}
```

```
boolean pesquisar(int x, No i) {
```

```
    boolean resp;  
    if (i == null) {  
        resp = false;  
    } else if (x == i.elemento) {  
        resp = true;  
    } else if (x < i.elemento) {  
        resp = pesquisar(x, i.esq);  
    } else {  
        resp = pesquisar(x, i.dir);  
    }  
    return resp;  
}
```



PESQUISAR

```
//Pesquisar(4)
```

```
boolean pesquisar(int x) {  
    return pesquisar(x, raiz);  
}
```

```
boolean pesquisar(int x, No i) {  
    boolean resp;
```

```
    if (i == null) {
```

```
        resp = false;
```

```
    } else if (x == i.elemento) {
```

```
        resp = true;
```

```
    } else if (x < i.elemento) {
```

```
        resp = pesquisar(x, i.esq);
```

```
    } else {
```

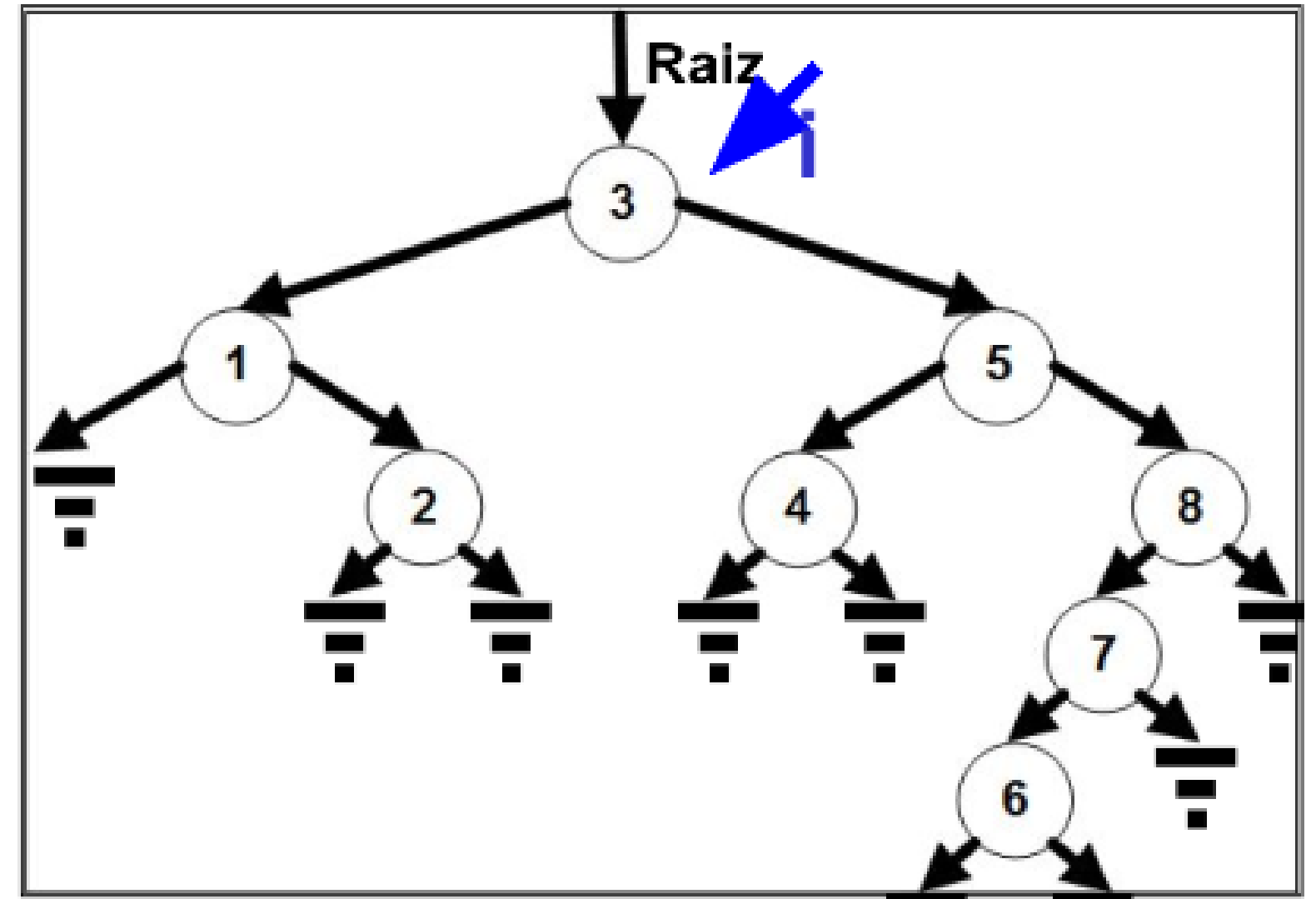
```
        resp = pesquisar(x, i.dir);
```

```
    }
```

```
    return resp;
```

```
}
```

false: n(3) == null



PESQUISAR

```
//Pesquisar(4)
```

```
boolean pesquisar(int x) {  
    return pesquisar(x, raiz);  
}
```

```
boolean pesquisar(int x, No i) {  
    boolean resp;  
    if (i == null) {  
        resp = false;
```

```
    } else if (x == i.elemento) {
```

```
        resp = true;
```

```
    } else if (x < i.elemento) {
```

```
        resp = pesquisar(x, i.esq);
```

```
    } else {
```

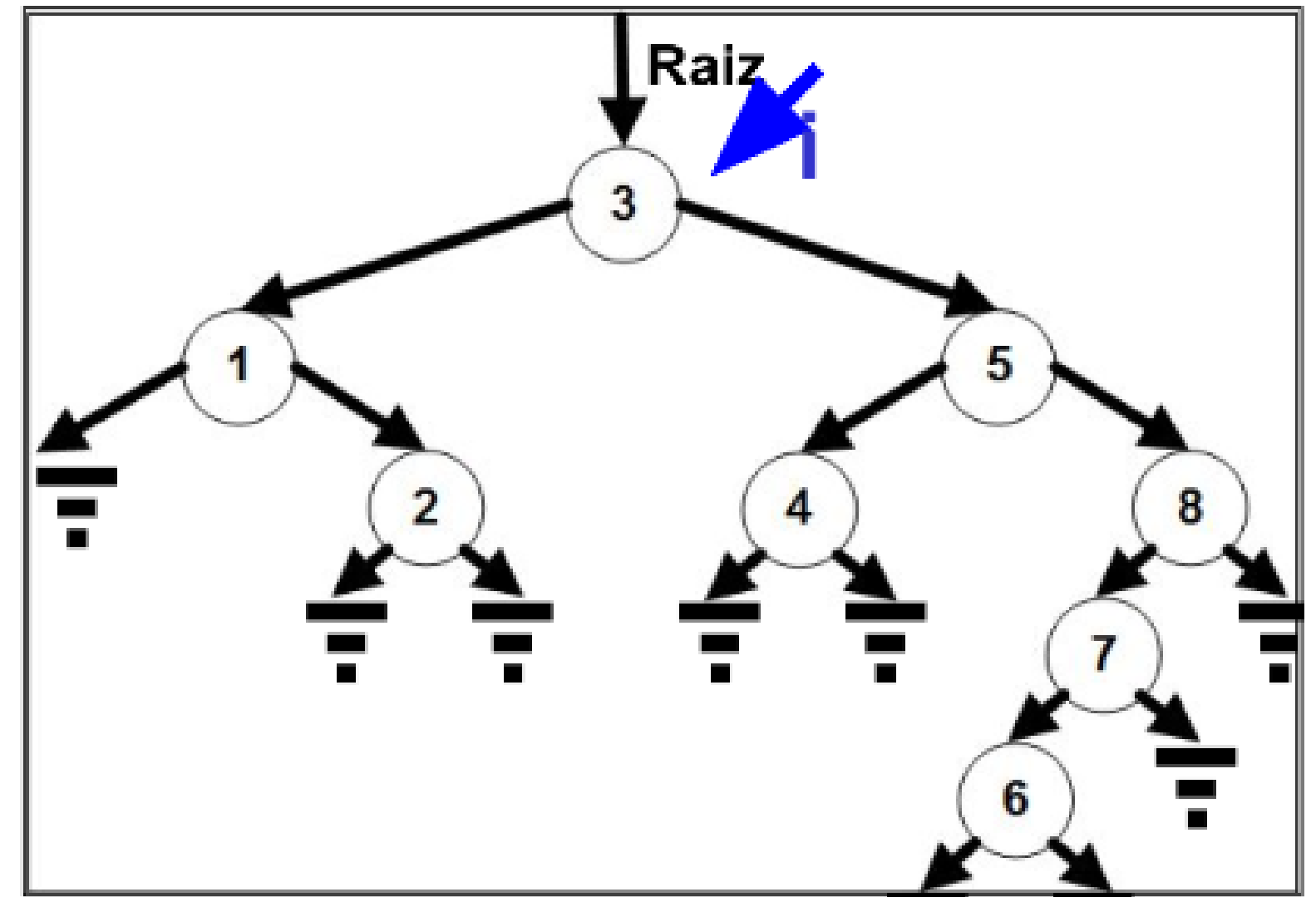
```
        resp = pesquisar(x, i.dir);
```

```
    }
```

```
    return resp;
```

```
}
```

false: 4 == 3



PESQUISAR

```
//Pesquisar(4)
```

```
boolean pesquisar(int x) {  
    return pesquisar(x, raiz);  
}
```

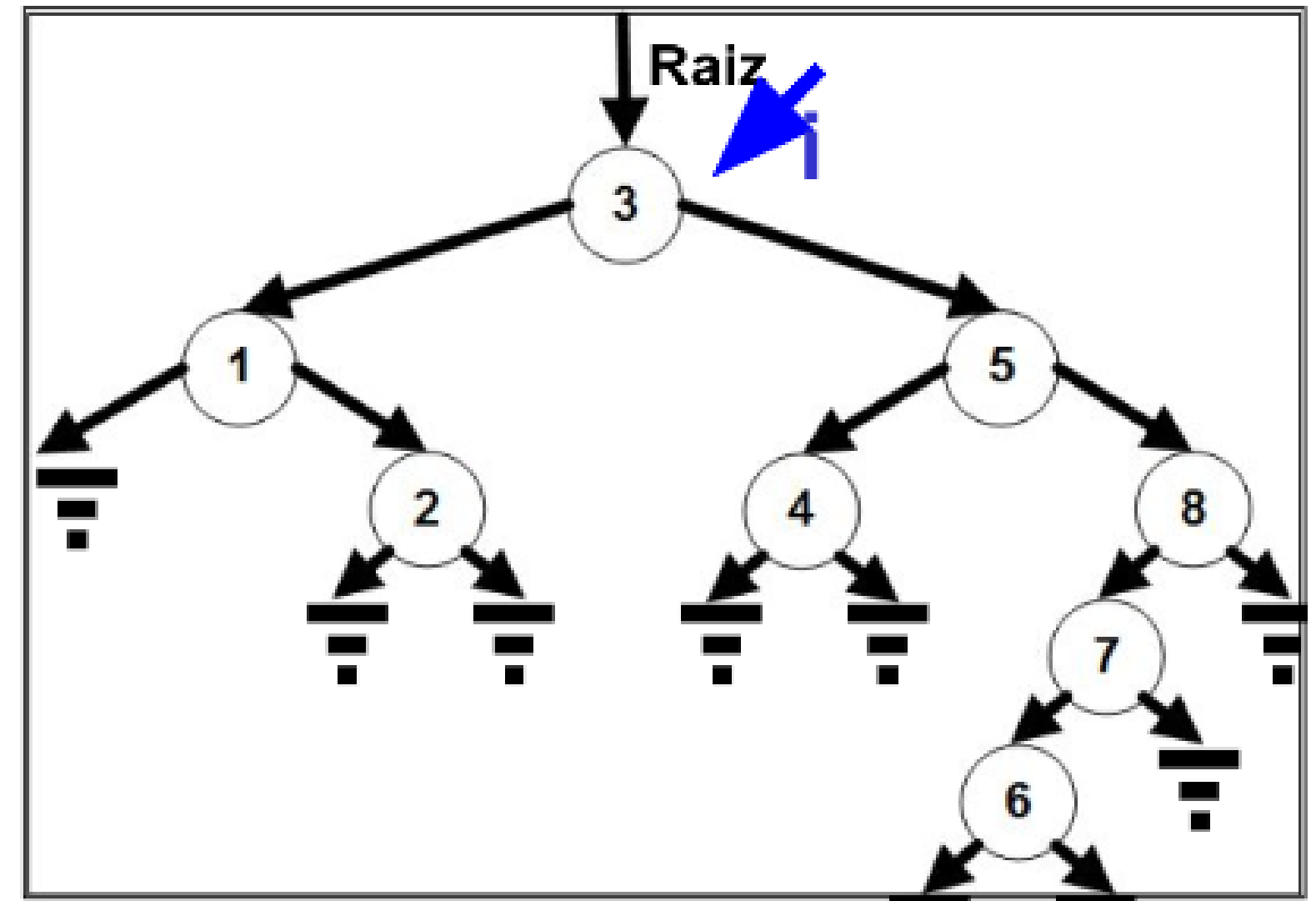
```
boolean pesquisar(int x, No i) {  
    boolean resp;  
    if (i == null) {  
        resp = false;  
    } else if (x == i.elemento) {  
        resp = true;
```

```
    } else if (x < i.elemento) {
```

```
        resp = pesquisar(x, i.esq);  
    } else {  
        resp = pesquisar(x, i.dir);  
    }
```

```
    return resp;
```

```
} false: 4 < 3
```

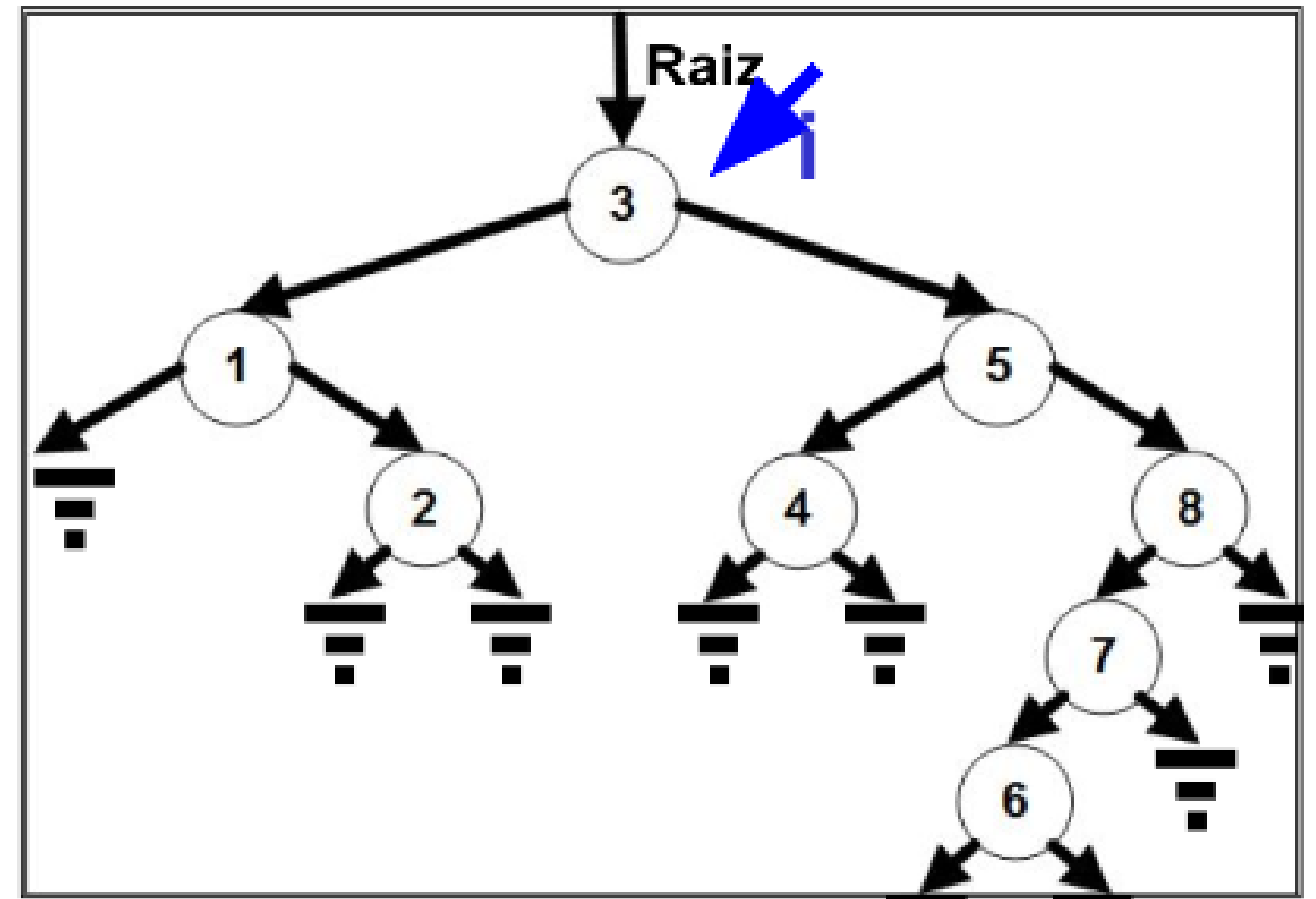


PESQUISAR

```
//Pesquisar(4)
```

```
boolean pesquisar(int x) {  
    return pesquisar(x, raiz);  
}
```

```
boolean pesquisar(int x, No i) {  
    boolean resp;  
    if (i == null) {  
        resp = false;  
    } else if (x == i.elemento) {  
        resp = true;  
    } else if (x < i.elemento) {  
        resp = pesquisar(x, i.esq);  
    } else {  
        resp = pesquisar(x, i.dir);  
    }  
    return resp;  
}
```



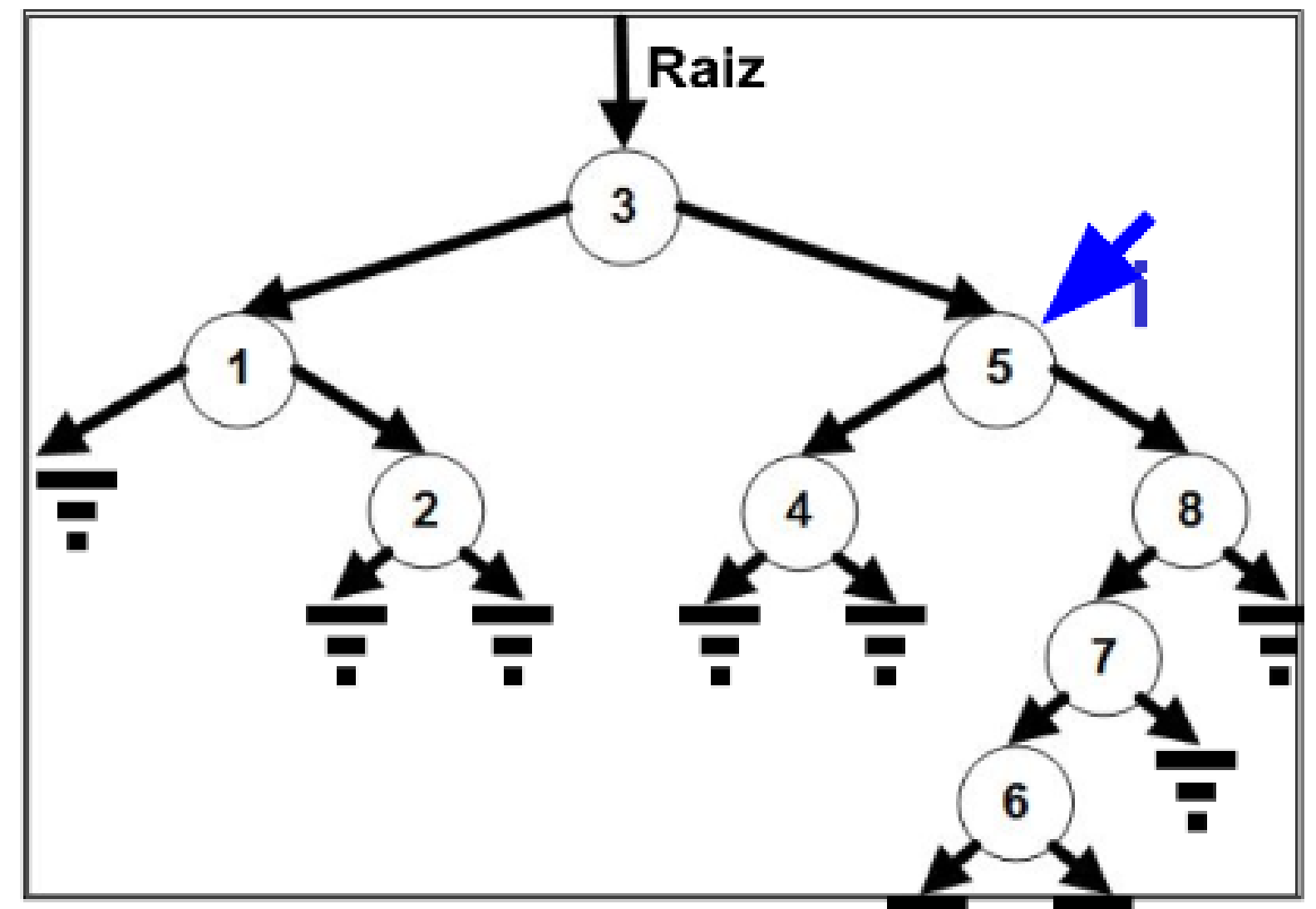
PESQUISAR

```
//Pesquisar(4)
```

```
boolean pesquisar(int x) {  
    return pesquisar(x, raiz);  
}
```

```
boolean pesquisar(int x, No i) {
```

```
    boolean resp;  
    if (i == null) {  
        resp = false;  
    } else if (x == i.elemento) {  
        resp = true;  
    } else if (x < i.elemento) {  
        resp = pesquisar(x, i.esq);  
    } else {  
        resp = pesquisar(x, i.dir);  
    }  
    return resp;  
}
```



PESQUISAR

```
//Pesquisar(4)
```

```
boolean pesquisar(int x) {  
    return pesquisar(x, raiz);  
}
```

```
boolean pesquisar(int x, No i) {  
    boolean resp;
```

```
    if (i == null) {
```

```
        resp = false;
```

```
    } else if (x == i.elemento) {
```

```
        resp = true;
```

```
    } else if (x < i.elemento) {
```

```
        resp = pesquisar(x, i.esq);
```

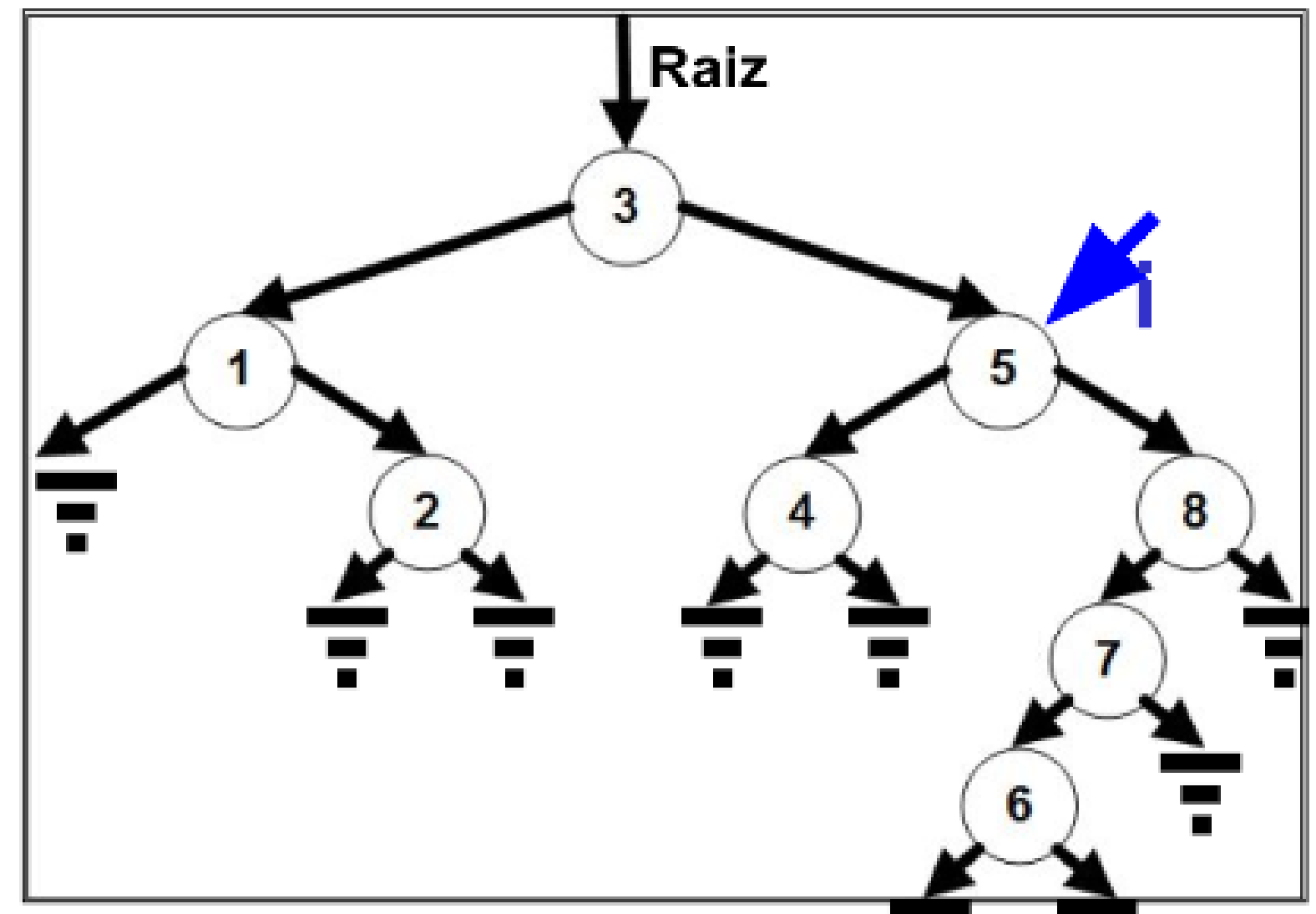
```
    } else {
```

```
        resp = pesquisar(x, i.dir);
```

```
    }
```

```
    return resp;
```

```
    }  
    false: n(5) == null
```



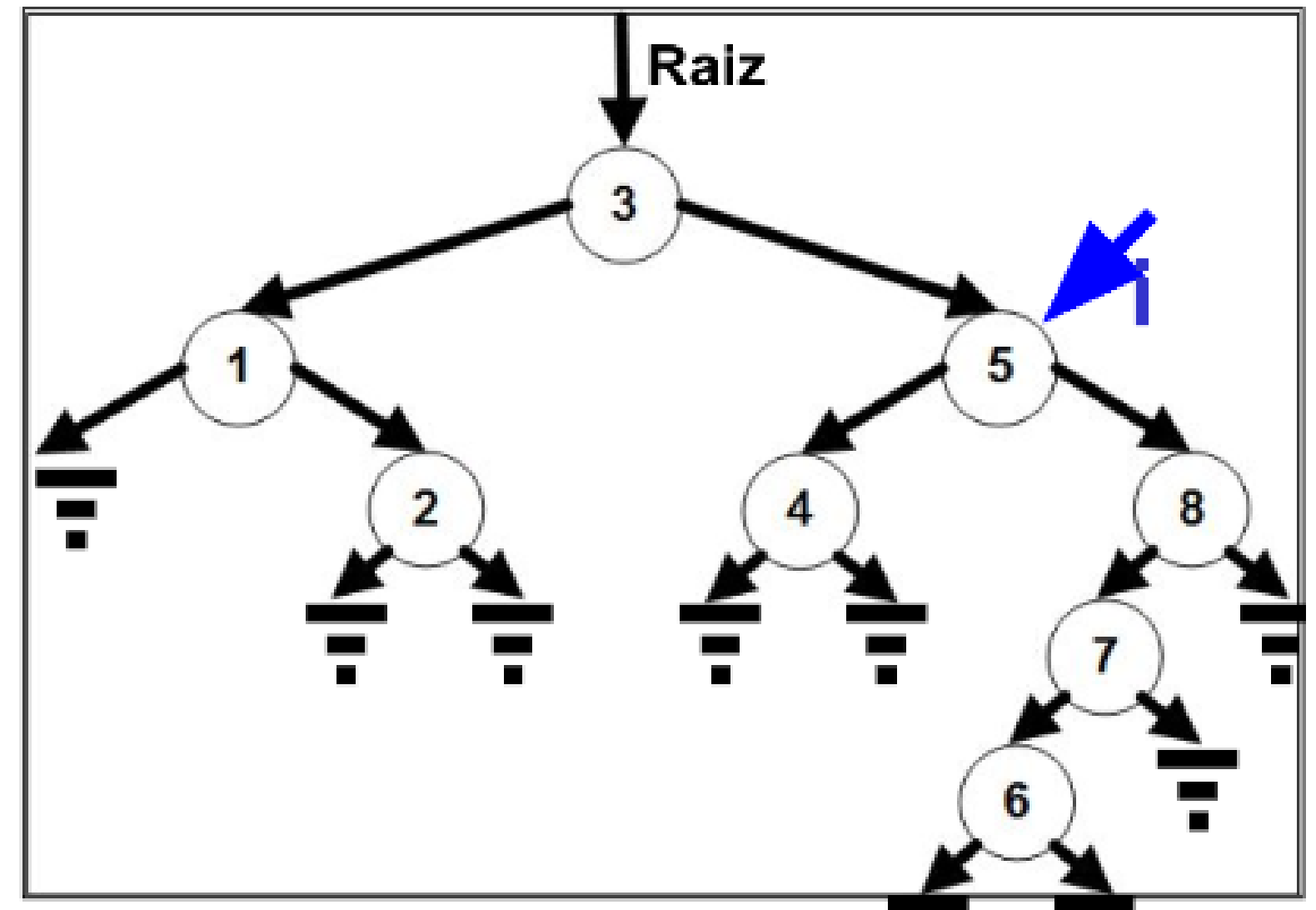
PESQUISAR

```
//Pesquisar(4)
```

```
boolean pesquisar(int x) {  
    return pesquisar(x, raiz);  
}
```

```
boolean pesquisar(int x, No i) {  
    boolean resp;  
    if (i == null) {  
        resp = false;  
    } else if (x == i.elemento) {  
        resp = true;  
    } else if (x < i.elemento) {  
        resp = pesquisar(x, i.esq);  
    } else {  
        resp = pesquisar(x, i.dir);  
    }  
    return resp;  
}
```

false: 4 == 5



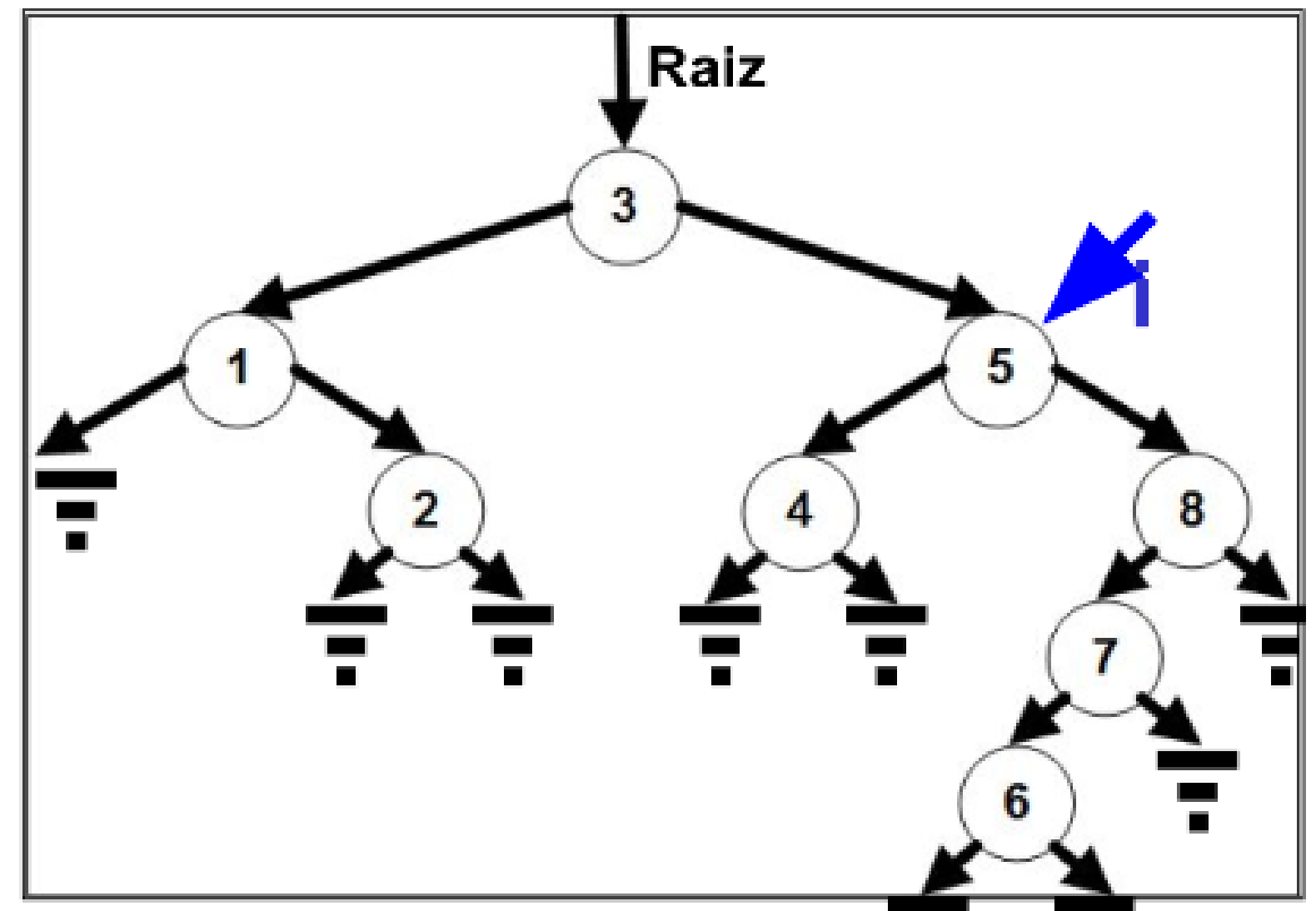
PESQUISAR

```
//Pesquisar(4)
```

```
boolean pesquisar(int x) {  
    return pesquisar(x, raiz);  
}
```

```
boolean pesquisar(int x, No i) {  
    boolean resp;  
    if (i == null) {  
        resp = false;  
    } else if (x == i.elemento) {  
        resp = true;  
    } else if (x < i.elemento) {  
        resp = pesquisar(x, i.esq);  
    } else {  
        resp = pesquisar(x, i.dir);  
    }  
    return resp;  
}
```

true: 4 < 5

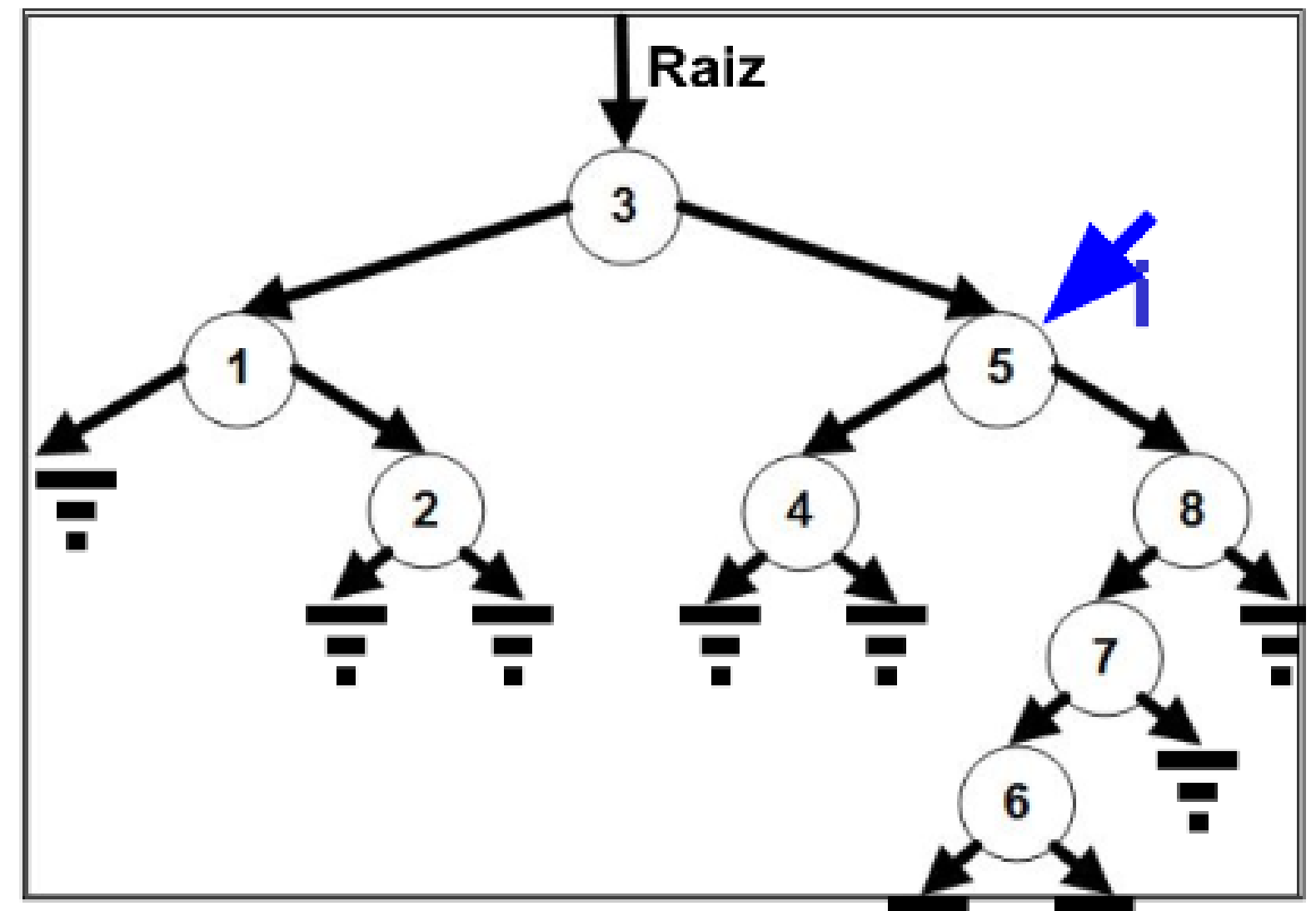


PESQUISAR

```
//Pesquisar(4)
```

```
boolean pesquisar(int x) {  
    return pesquisar(x, raiz);  
}
```

```
boolean pesquisar(int x, No i) {  
    boolean resp;  
    if (i == null) {  
        resp = false;  
    } else if (x == i.elemento) {  
        resp = true;  
    } else if (x < i.elemento) {  
        resp = pesquisar(x, i.esq);  
    } else {  
        resp = pesquisar(x, i.dir);  
    }  
    return resp;  
}
```



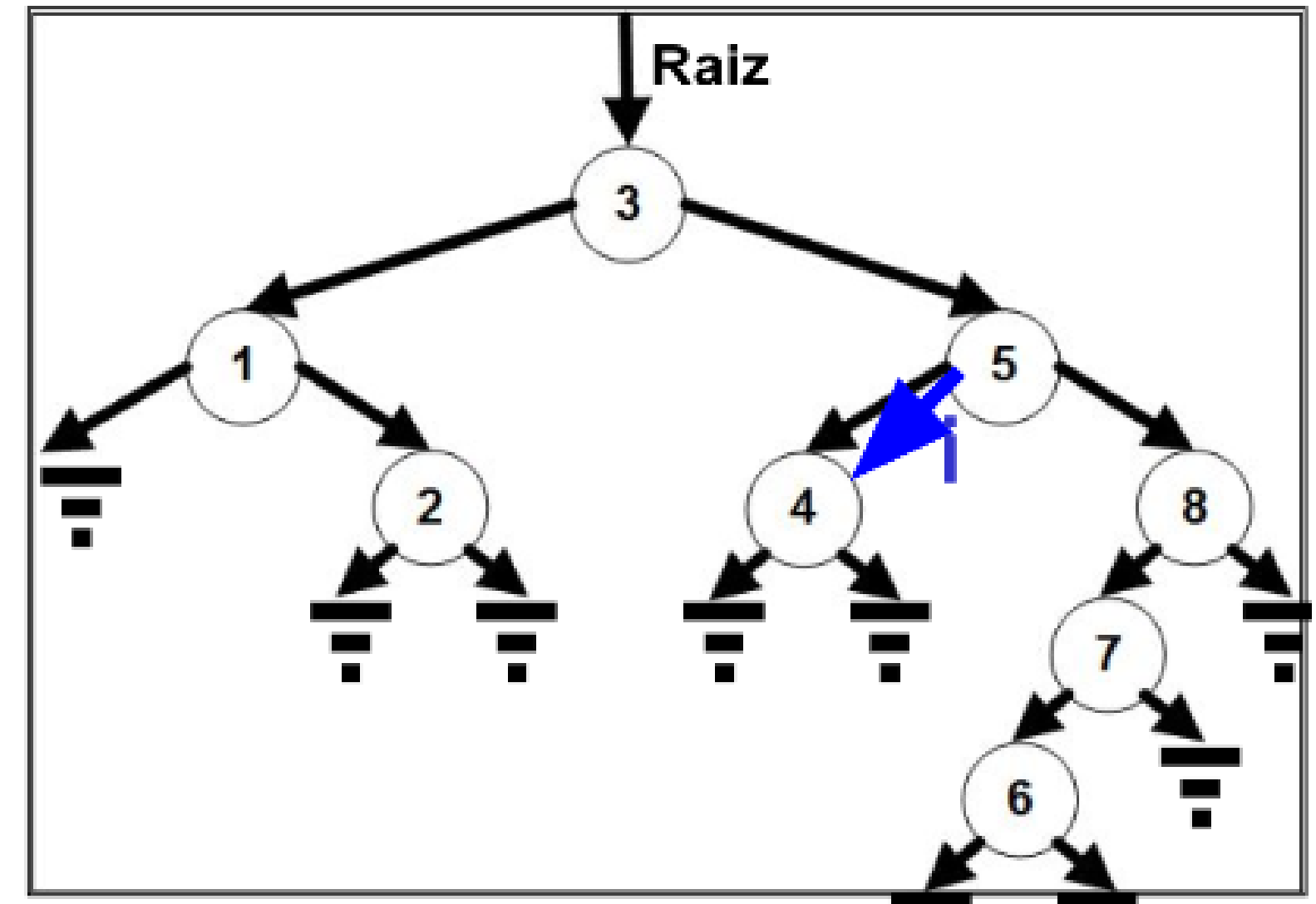
PESQUISAR

```
//Pesquisar(4)
```

```
boolean pesquisar(int x) {  
    return pesquisar(x, raiz);  
}
```

```
boolean pesquisar(int x, No i) {
```

```
    boolean resp;  
    if (i == null) {  
        resp = false;  
    } else if (x == i.elemento) {  
        resp = true;  
    } else if (x < i.elemento) {  
        resp = pesquisar(x, i.esq);  
    } else {  
        resp = pesquisar(x, i.dir);  
    }  
    return resp;  
}
```



PESQUISAR

```
//Pesquisar(4)
```

```
boolean pesquisar(int x) {  
    return pesquisar(x, raiz);  
}
```

```
boolean pesquisar(int x, No i) {  
    boolean resp;
```

```
    if (i == null) {
```

```
        resp = false;
```

```
    } else if (x == i.elemento) {
```

```
        resp = true;
```

```
    } else if (x < i.elemento) {
```

```
        resp = pesquisar(x, i.esq);
```

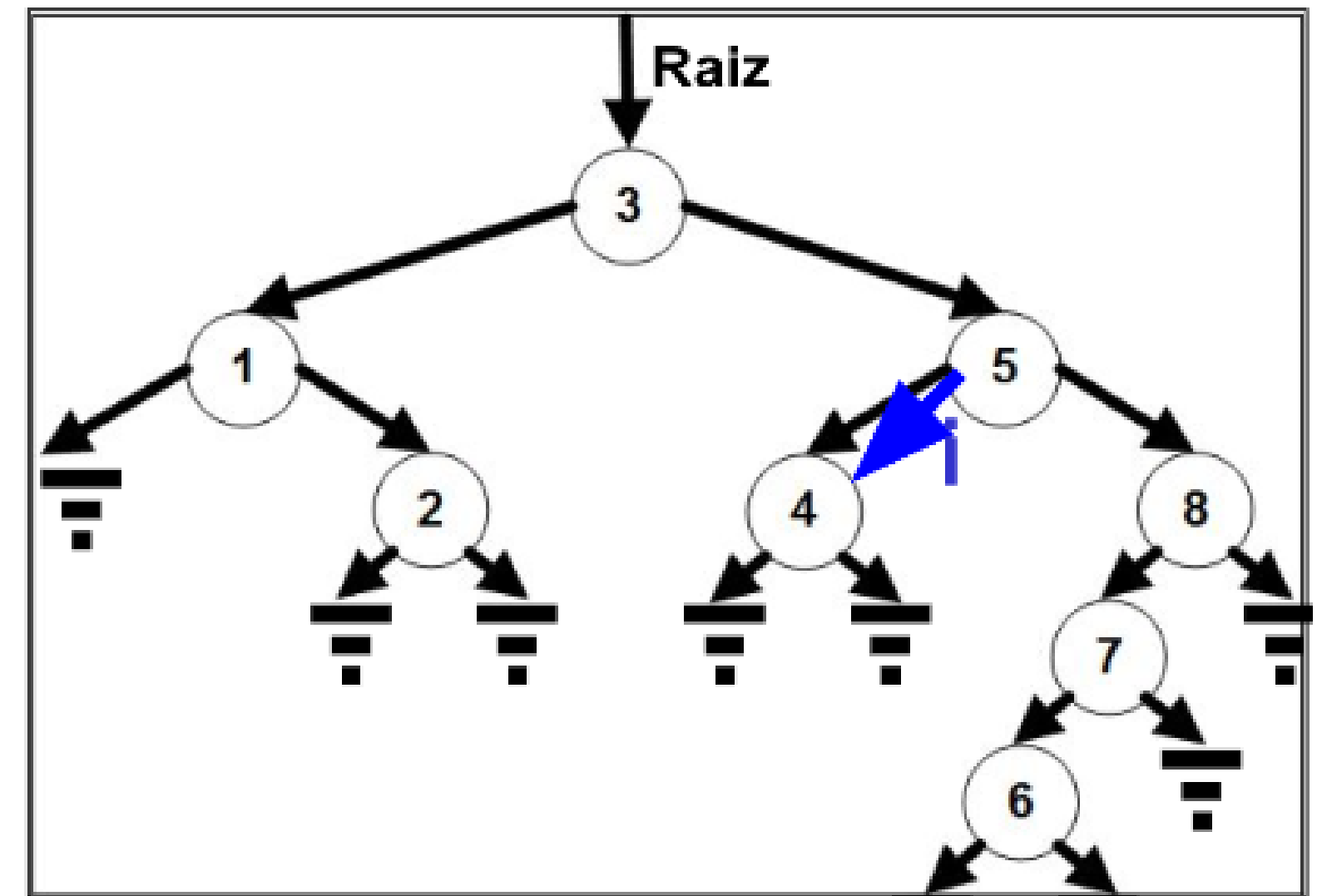
```
    } else {
```

```
        resp = pesquisar(x, i.dir);
```

```
    }
```

```
    return resp;
```

```
} false: n(4) == null
```



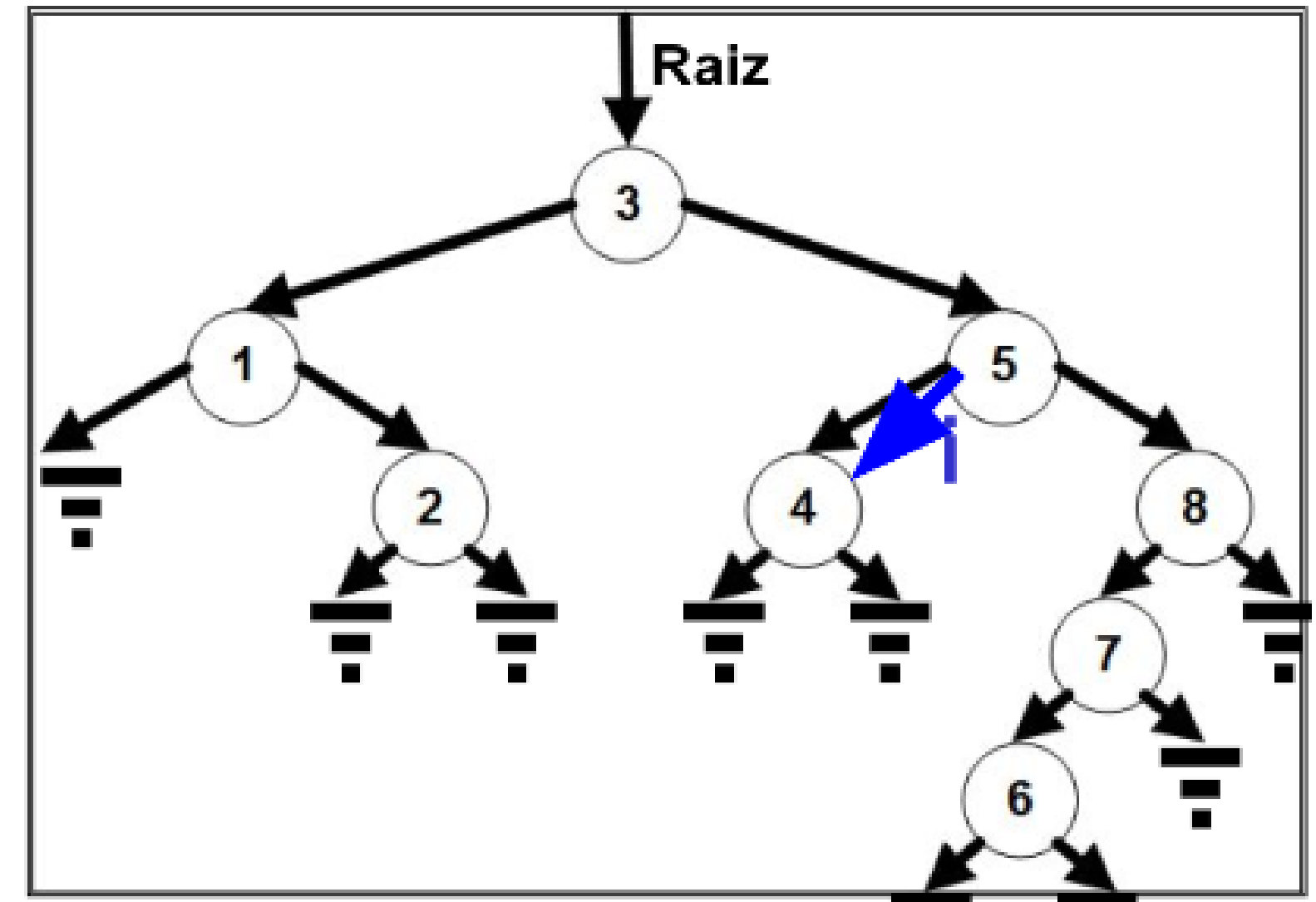
PESQUISAR

```
//Pesquisar(4)
```

```
boolean pesquisar(int x) {  
    return pesquisar(x, raiz);  
}
```

```
boolean pesquisar(int x, No i) {  
    boolean resp;  
    if (i == null) {  
        resp = false;  
    } else if (x == i.elemento) {  
        resp = true;  
    } else if (x < i.elemento) {  
        resp = pesquisar(x, i.esq);  
    } else {  
        resp = pesquisar(x, i.dir);  
    }  
    return resp;  
}
```

true: 4 == 4



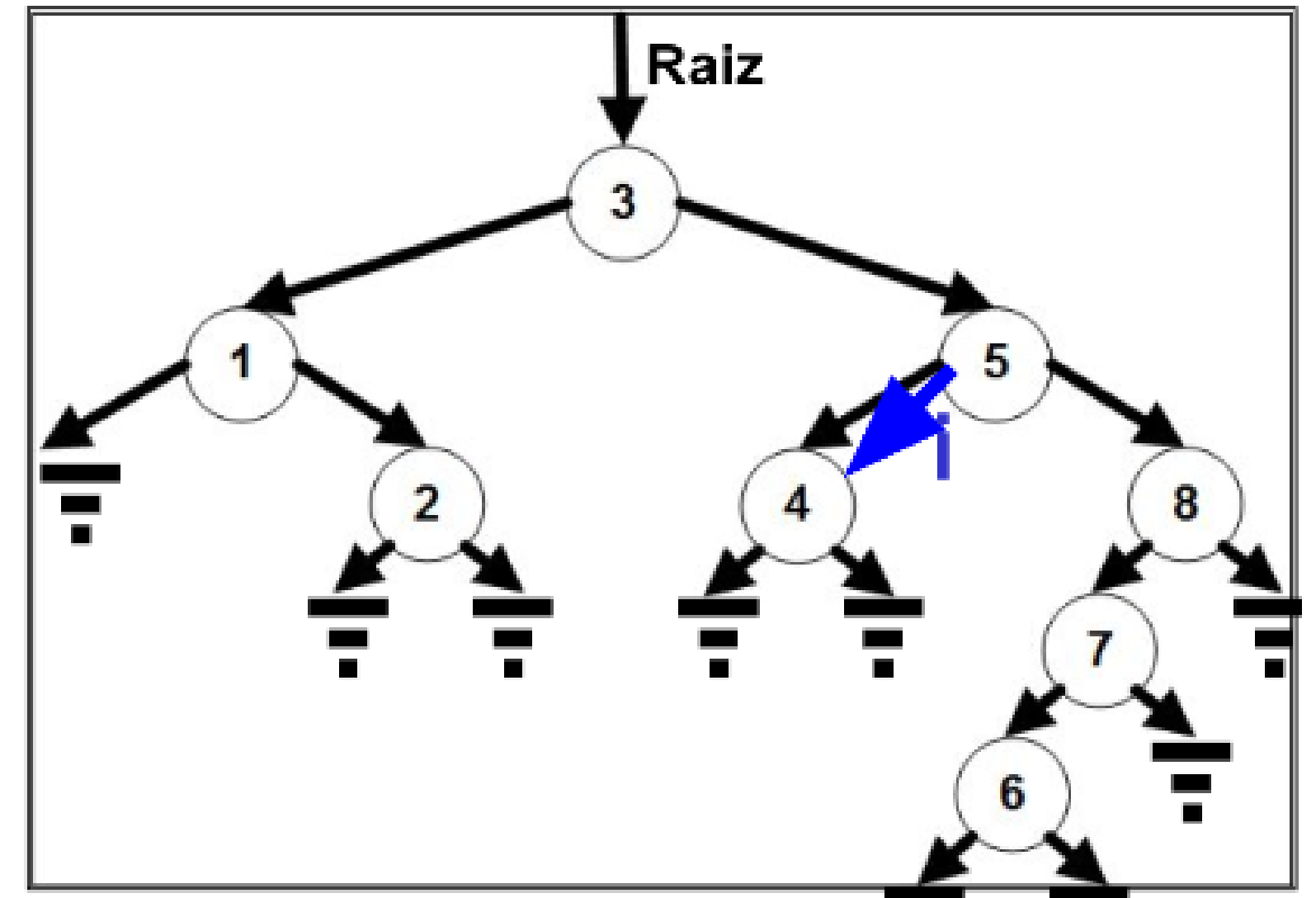
PESQUISAR

//Pesquisar(4)

```
boolean pesquisar(int x) {  
    return pesquisar(x, raiz);  
}
```

```
boolean pesquisar(int x, No i) {  
    boolean resp;  
    if (i == null) {  
        resp = false;  
    } else if (x == i.elemento) {  
        resp = true;
```

```
    } else if (x < i.elemento) {  
        resp = pesquisar(x, i.esq);  
    } else {  
        resp = pesquisar(x, i.dir);  
    }  
    return resp;  
}
```



PESQUISAR

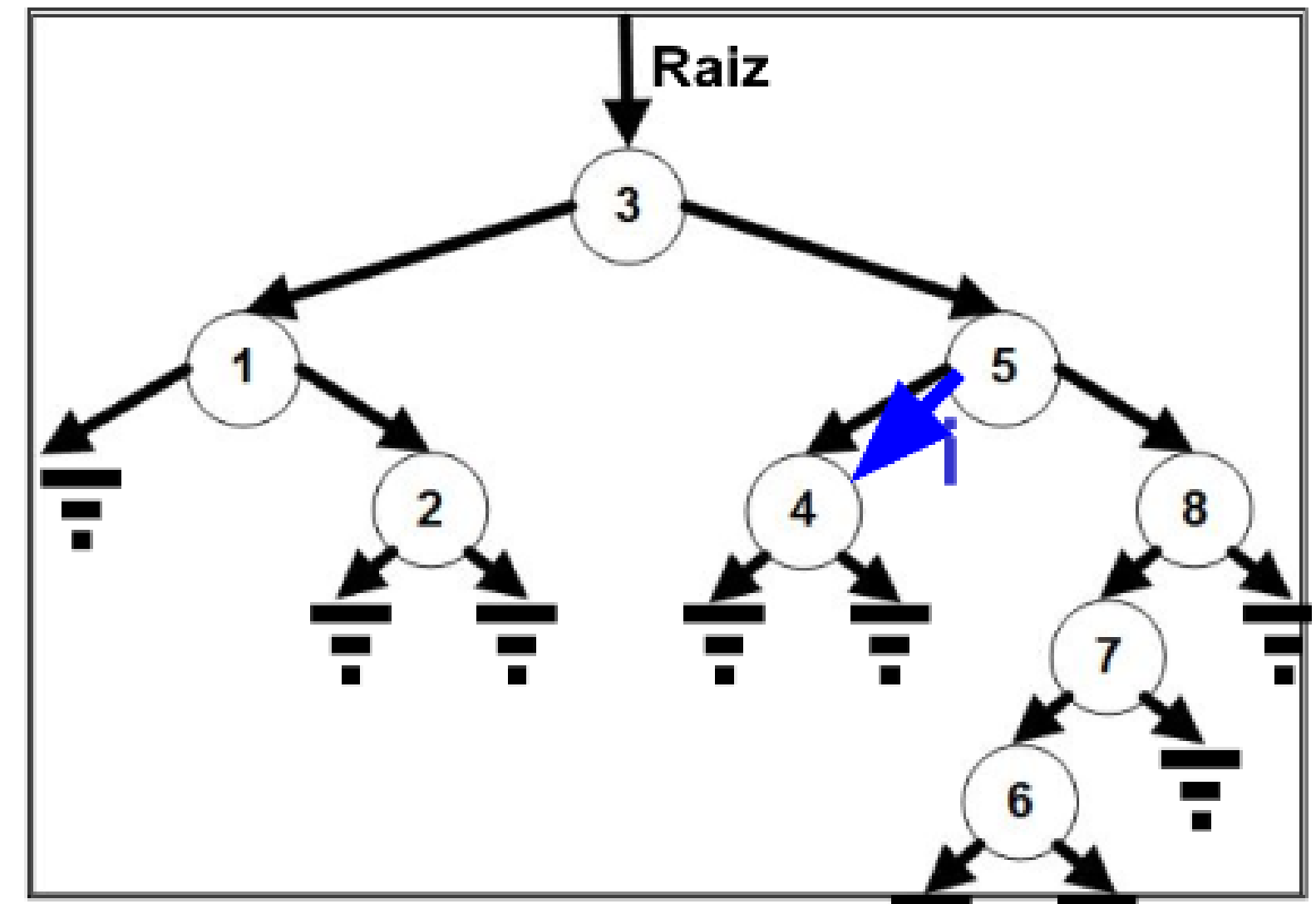
```
//Pesquisar(4)
```

```
boolean pesquisar(int x) {  
    return pesquisar(x, raiz);  
}
```

```
boolean pesquisar(int x, No i) {  
    boolean resp;  
    if (i == null) {  
        resp = false;  
    } else if (x == i.elemento) {  
        resp = true;  
    } else if (x < i.elemento) {  
        resp = pesquisar(x, i.esq);  
    } else {  
        resp = pesquisar(x, i.dir);  
    }  
}
```

```
return resp;
```

```
}
```



CAMINHAR

Caminhar Pre: Primeiro printa o elemento para depois caminhar. (Printa primeiro os pais).

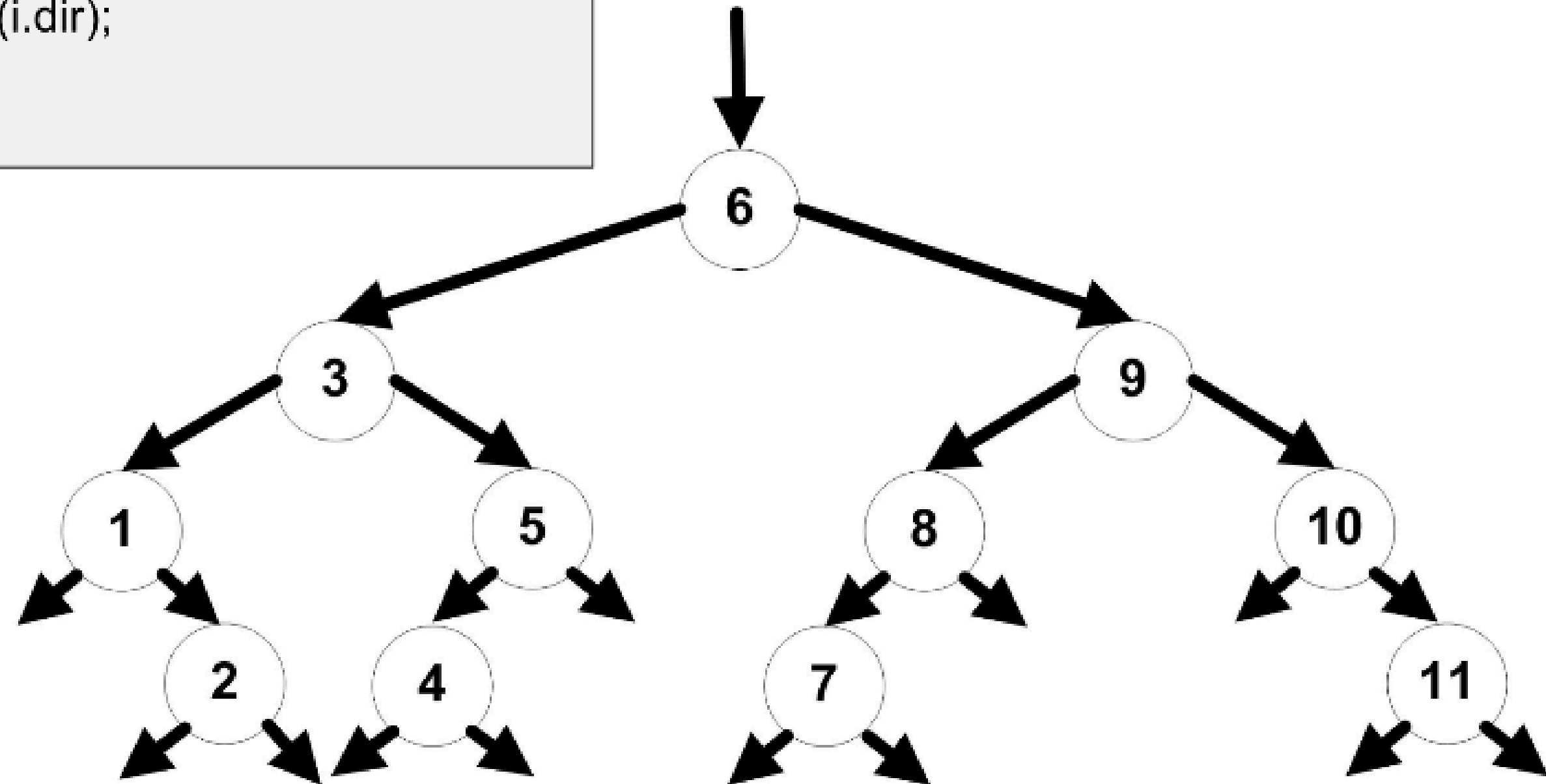
Caminhar Pos: Primeiro caminha para os dois lados até nulo para depois printar o elemento. (Printa primeiro os filhos).

Caminhar Central: Primeiro caminha para um lado até o nulo, após isso printa o elemento, e após caminha para a direita até o nulo. (Printa em ordem crescente, ou, decrescente, dependendo para qual lado caminhar).



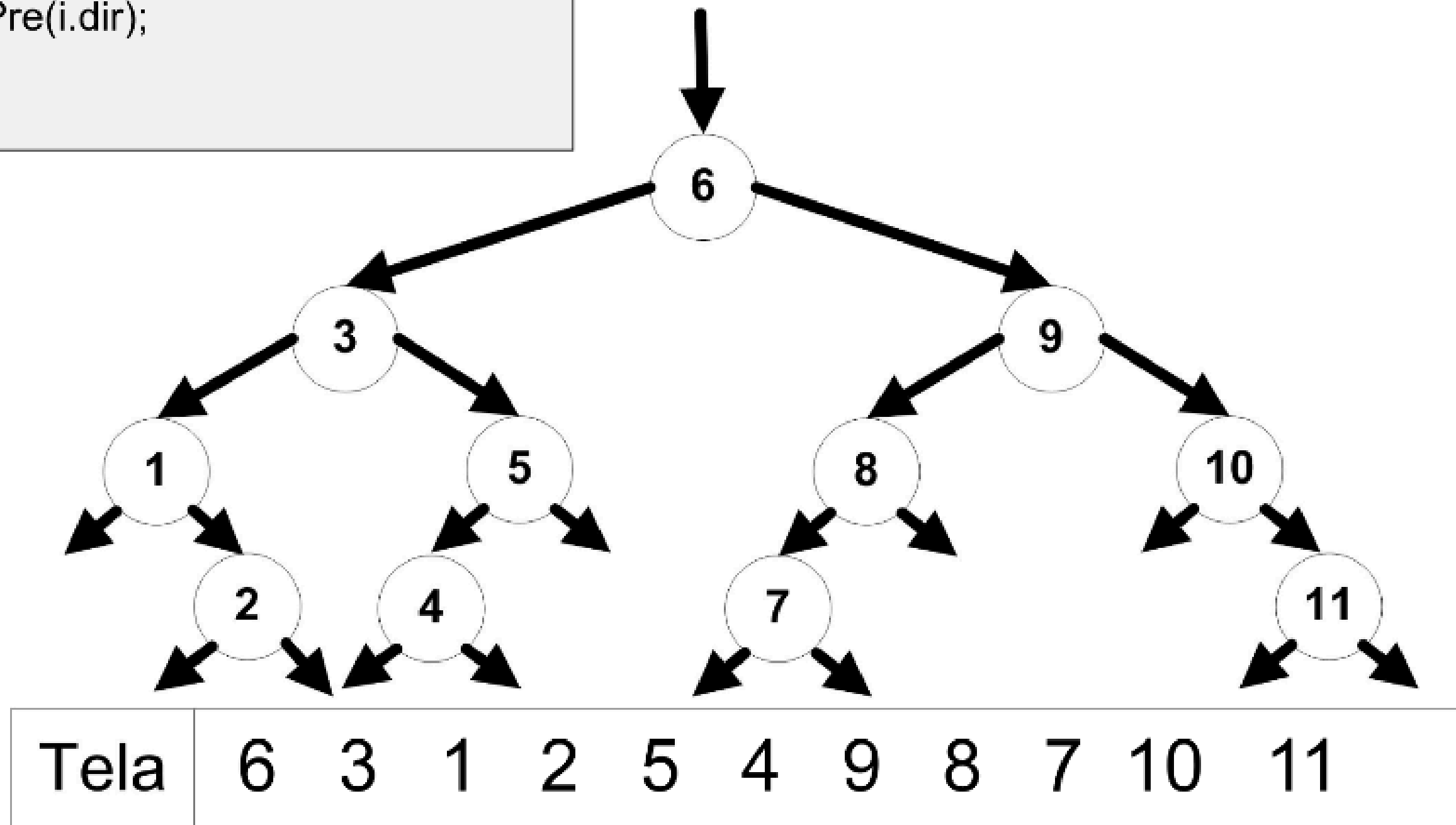
CAMINHAR PRE

```
void caminharPre(No i) {  
    if (i != null) {  
        System.out.print(i.elemento + " ");  
        caminharPre(i.esq);  
        caminharPre(i.dir);  
    }  
}
```



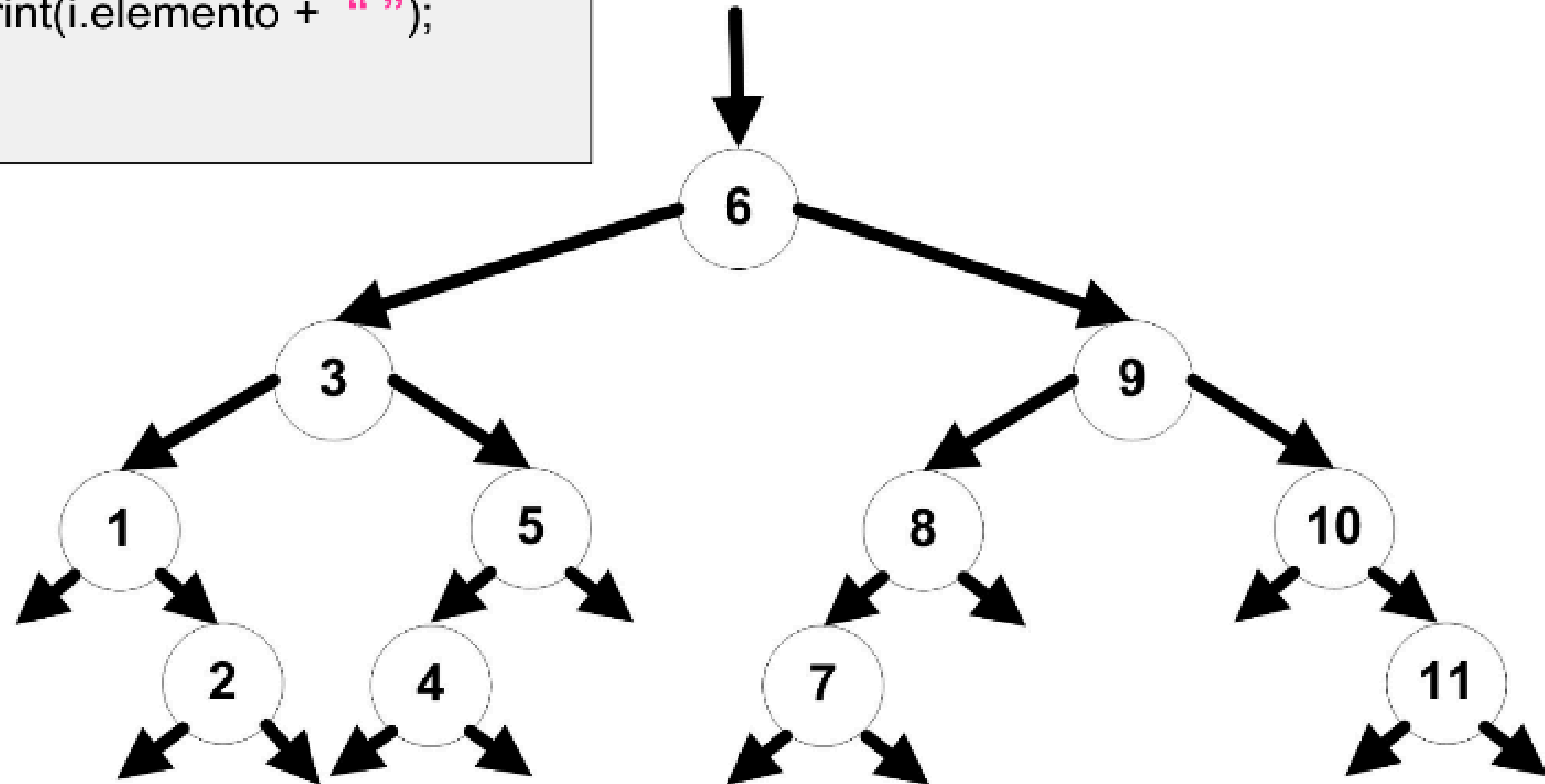
CAMINHAR PRE

```
void caminharPre(No i) {  
    if (i != null) {  
        System.out.print(i.elemento + " ");  
        caminharPre(i.esq);  
        caminharPre(i.dir);  
    }  
}
```



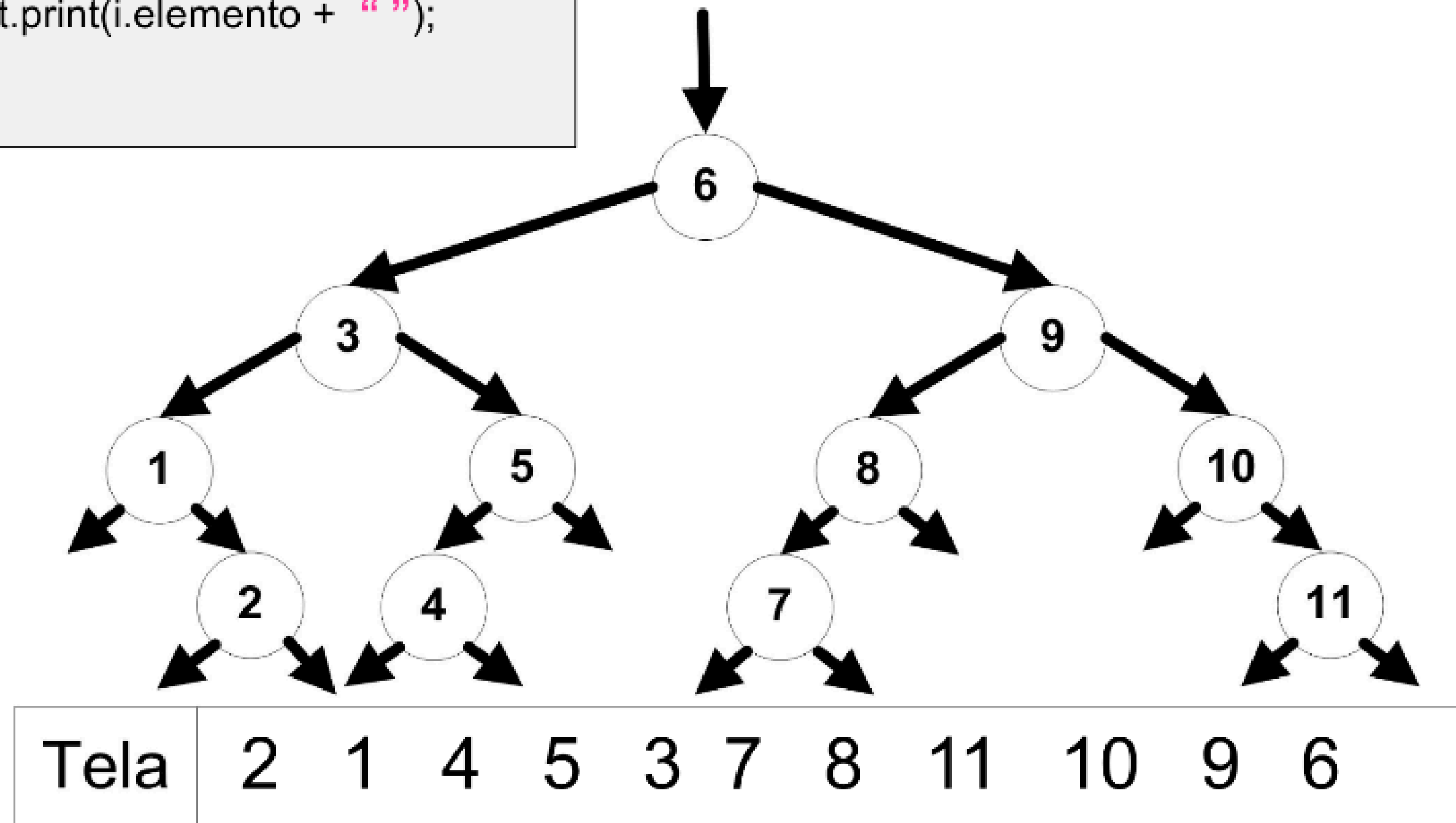
CAMINHAR POS

```
void caminharPos(No i) {  
    if (i != null) {  
        caminharPos(i.esq);  
        caminharPos(i.dir);  
        System.out.print(i.elemento + " ");  
    }  
}
```



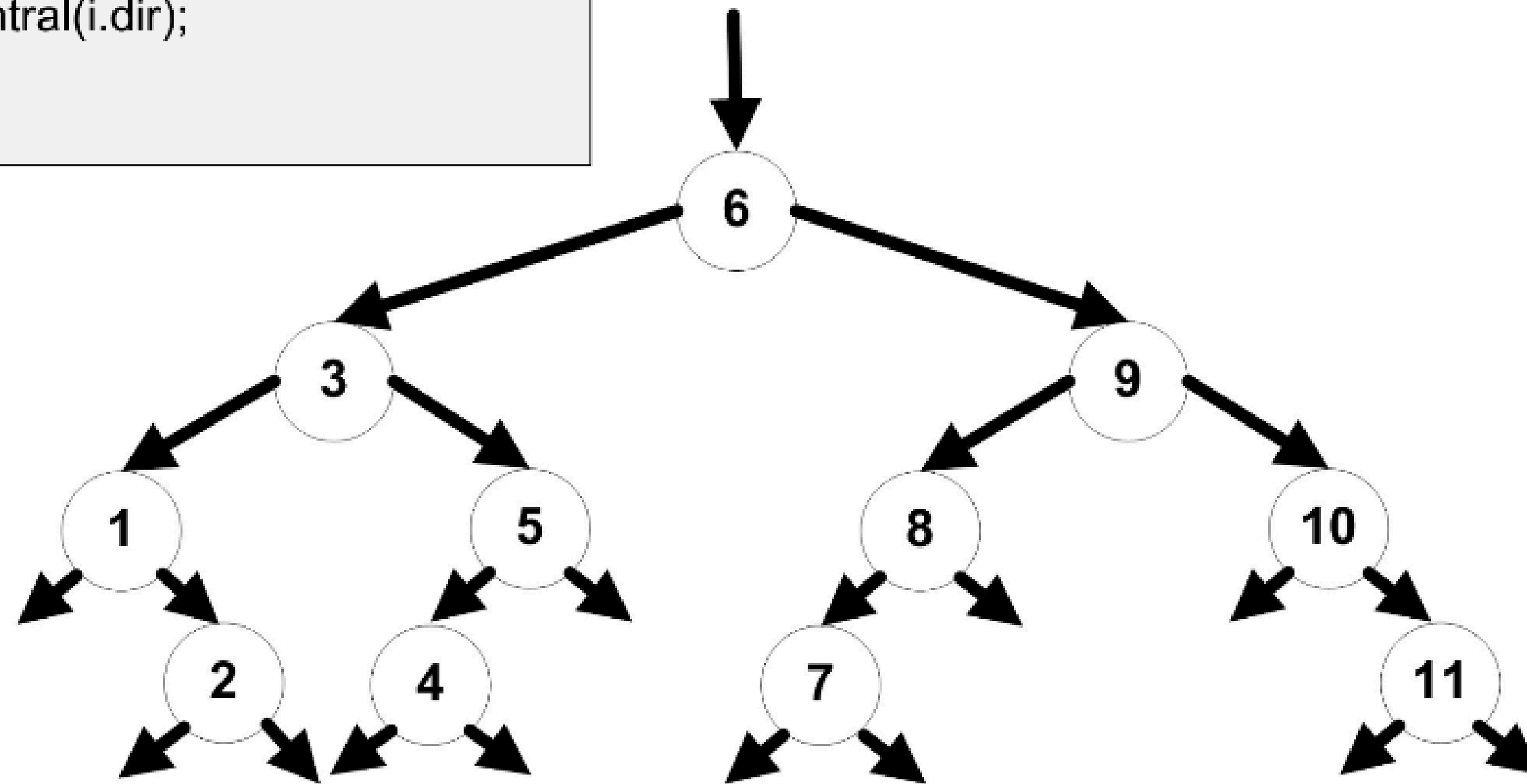
CAMINHAR POS

```
void caminharPos(No i) {  
    if (i != null) {  
        caminharPos(i.esq);  
        caminharPos(i.dir);  
        System.out.print(i.elemento + " ");  
    }  
}
```



CAMINHAR CENTRAL

```
void caminharCentral(No i) {  
    if (i != null) {  
        caminharCentral(i.esq);  
        System.out.print(i.elemento + " ");  
        caminharCentral(i.dir);  
    }  
}
```



CAMINHAR CENTRAL

```
void caminharCentral(No i) {  
    if (i != null) {  
        caminharCentral(i.esq);  
        System.out.print(i.elemento + " ");  
        caminharCentral(i.dir);  
    }  
}
```

